

Quintal em Guarda

Proposta de jogo 2D para Roblox

Game design, especificação para Claude Code e direção de arte

Preparado para Breno • Versão 1.1 • 15 de setembro de 2026



Conceito original de mundo e personagens. Referência visual, sem jogo implementado.

Recomendação: um tower defense cooperativo de brinquedos vivos, com seis torres colecionáveis, três mapas e Circuitos entre personagens vizinhos. O documento transforma referências atuais do gênero em regras, conteúdo, arte e critérios de construção.

A versão 1.1 inclui 40 PNGs individuais, índice de assets, plano de animação e instruções para o Claude Code. Balanceamento e metas de produto são hipóteses para testar.

Como usar este documento

Para revisar a proposta, comece pelo capítulo 1. Para construir, leia também o capítulo 2 e use os dados do capítulo 6. Para produzir imagens e interface, siga o capítulo 3. O capítulo 5 inicia a implementação. O capítulo 8 explica os 40 PNGs entregues, sua localização e as animações.

Capítulo	Conteúdo	Página
1 Proposta e game design	Referências, público, regras, progressão e escopo	3
2 Especificação técnica	Arquitetura 2D, rede, dados, combate e integração	11
3 Direção de arte	Personagens, cenários, interface e assets	18
4 Testes e aceite	Cenários e definição de pronto	25
5 Instruções para Claude Code	Mensagem de início e retomada	27
6 Catálogo numérico	Torres, inimigos, ondas e coordenadas	29
7 Fontes e pesquisa	Links consultados e limites da análise	33
8 Integração das imagens e animações	Arquivos, upload, apoios, movimento e produção	34

Premissas da proposta

Adotamos tabuleiro 2D, personagens originais cartoon, foco em computador e celular e partidas curtas. Orçamento, equipe e cronograma não foram informados; os marcos indicam sequência de trabalho e não uma promessa de prazo.

A pesquisa reúne cinco experiências do Roblox e três referências externas. A seleção considera utilidade para o projeto, sem apresentar um ranking absoluto de audiência. Não houve playtest próprio dos concorrentes.

Este documento especifica um jogo a construir. Publicação, testes reais, moderação e integração de assets continuam sendo etapas de produção. O catálogo possui números iniciais que precisam de validação.

1 Proposta e game design

Proposta de jogo 2D para Roblox

Versão 1.1 de 15 de setembro de 2026. Preparado para Breno e para implementação pelo Claude Code. O nome é provisório. Este documento define um produto inicial completo e publicável após implementação, produção de assets e validação. Nenhum jogo foi implementado ou testado neste trabalho.

Recomendação: criar um tower defense cooperativo de brinquedos vivos, com partidas de 10 a 15 minutos, visão superior em 2D, seis torres colecionáveis e combinações por proximidade chamadas Circuitos. O jogador monta uma pequena equipe, posiciona seus personagens e escolhe especializações durante a partida para proteger um farol de brinquedo contra criaturas de poeira.

A aposta de produto é tornar a estratégia legível no celular e dar identidade aos personagens. A satisfação deve vir de perceber que uma combinação funcionou, ver uma torre mudar de aparência e terminar uma defesa com amigos. A hipótese ainda precisa ser validada com jogadores; pesquisa de jogos existentes não comprova retenção ou receita deste projeto.

Decisões de partida

Tema	Decisão para a versão 1
Plataforma	Roblox com Luau e Rojo
Apresentação	Tabuleiro e personagens 2D em ScreenGui
Público de projeto	Jogadores casuais e intermediários, familiaridade variável com TD
Dispositivos	Computador, celular e tablet em paisagem
Grupo	Solo ou cooperação de 2 a 4 pessoas
Conteúdo	Tutorial, treino, 3 mapas, 20 ondas, 6 torres, 8 inimigos e 3 chefes
Progressão	Desbloqueios determinísticos de personagens e domínio cosmético
Monetização	Cosméticos opcionais, inicialmente um passe permanente
Idiomas	Português brasileiro e inglês desde a estrutura inicial
Principal diferencial	Três Circuitos visíveis entre pares de torres adjacentes

Referências atuais e método de seleção

“Melhores” significa aqui as referências mais úteis para este projeto: profundidade estratégica, identidade, cooperação, progressão e capacidade de ensinar uma mecânica. Não é um ranking absoluto de popularidade. A pesquisa foi realizada em 15 de setembro de 2026 com páginas oficiais dos jogos, páginas dos desenvolvedores e documentação da plataforma. Não houve sessão prática nesses jogos nem acesso às suas métricas internas.

As páginas do Roblox podem apresentar conteúdo carregado por JavaScript e textos genéricos de ausência de servidores. Esse texto não foi interpretado como encerramento dos jogos. Números indexados de jogadores simultâneos não foram usados como medição em tempo real. As conclusões abaixo distinguem características anunciadas dos jogos e nossas decisões de design.

Tower Defense Simulator

A página oficial apresenta defesa por unidades, cooperação contra chefes e desbloqueio de novas torres. Na consulta, anunciava a versão 2.10.0 e a unidade Pulse Trooper. É a referência principal para o ciclo de melhorar a coleção e enfrentar desafios com amigos. [Página oficial de Tower Defense Simulator](#).

Aplicação proposta: deixar a próxima torre e seu custo sempre visíveis; apresentar um chefe como objetivo reconhecível; permitir repetir uma partida com o mesmo grupo em um toque. Risco a evitar no nosso projeto:

esconder a diversão inicial atrás de muitas partidas repetidas. O tutorial já entrega três funções complementares.

Tower Defense X

A descrição oficial destaca torres de ar, terra e mar, chefes capazes de destruir torres e um modo PvP. Serve como referência de ameaças que exigem adaptação, além de simplesmente elevar a vida dos inimigos. [Página oficial de Tower Defense X](#).

Aplicação proposta: três chefes com comportamentos legíveis e contrajogo. Para o primeiro lançamento, ataques dos chefes podem reduzir temporariamente a cadência, mas não apagam torres compradas pelo jogador. Isso reduz a punição enquanto se aprende a economia. PvP fica para uma avaliação futura, pois exigiria outro balanceamento e outra estrutura de partidas.

Tower Heroes

A página oficial descreve heróis distintos, mapas com chefes próprios, campanha cooperativa e coleção de skins, adesivos e modificadores. É a principal referência de identidade e vínculo com o elenco. [Página oficial de Tower Heroes](#).

Aplicação proposta: seis silhuetas fáceis de reconhecer, reações expressivas e uma ficha de coleção por personagem. Cada mapa terá um chefe que pertence visualmente àquele lugar. Cosméticos precisam parecer desejáveis porque o personagem é querido, e essa aceitação deve ser medida no teste de conceito.

Anime Vanguard

A descrição oficial enfatiza invocar unidades, subir de nível, evoluí-las e jogar em grupo em diferentes modos. Sua utilidade para a proposta é a força da fantasia de coleção e transformação de personagens. [Página oficial de Anime Vanguard](#).

Aplicação proposta: upgrades com mudança visual forte e revelação curta ao desbloquear uma unidade. Não adotaremos personagens de franquias de anime nem rolagens aleatórias de poder. A coleção será original e terá preço conhecido em moeda obtida jogando. Isso também deixa o balanceamento e os testes reproduzíveis.

Doomspire Defense

A página oficial descreve defesa de uma torre, recompensas em moedas e experiência, compra de unidades e combinações de equipamentos com funções diferentes. É uma referência de como um tema familiar à cultura do Roblox pode dar coesão a um TD. [Página oficial de Doomspire Defense](#).

Aplicação proposta: formas de brinquedos, feedback imediato e humor visual. Cada nova torre deve abrir uma estratégia. Não precisamos reproduzir a estética de blocos ou o universo de Doomspire para aproveitar esse princípio.

Referências fora do Roblox

Bloons TD 6 combina caminhos de upgrade, heróis, desafios e cooperação para quatro jogadores. É a referência principal para escolhas de especialização e repetição com novas composições. Adaptaremos a clareza das funções para duas opções finais por torre, evitando oferecer muitas árvores de evolução ao iniciante. [Bloons TD 6 na Steam](#).

Kingdom Rush 5 Alliance trabalha com controle de dois heróis e campanha com desafios próprios por fase. A proposta aproveita a ideia de ameaças com identidade e encontros preparados. Não inclui herói móvel na versão 1, para concentrar os controles de celular em posicionar e melhorar torres. [Perguntas respondidas pelos desenvolvedores de Alliance](#).

Emberward usa blocos para construir labirintos e torres que tornam o posicionamento relevante. É a referência para fazer o tabuleiro importar. Adotaremos adjacência e Circuitos em caminhos fixos; construção livre de labirintos aumentaria a complexidade de navegação, cooperação e balanceamento. [Emberward na Steam](#).

Prioridade das referências para este projeto

Referência	Melhor aprendizado	Decisão concreta
Tower Heroes	Personalidade do elenco	Seis personagens com silhuetas e expressões próprias
Tower Defense Simulator	Ciclo de coleção e cooperação	Coleção visível e revanche com o grupo
Bloons TD 6	Especialização tática	Duas escolhas finais exclusivas por torre
Tower Defense X	Chefes alteram decisões	Três mecânicas anunciadas antes de acontecer
Anime Vanguard	Desejo de evoluir personagens	Transformação visual a cada estado
Doomspire Defense	Identidade coerente e composições	Tema de brinquedos e funções complementares
Kingdom Rush Alliance	Encontros com personalidade	Campanha curta com chefes distintos
Emberward	Posição é parte da estratégia	Circuitos por adjacência

Fantasia e experiência do jogador

O quintal fica vivo quando a casa dorme. Brinquedos esquecidos mantêm aceso o Farol de Pilha, enquanto Fiapos e criaturas do Breu tentam apagar sua luz. Os jogadores são os organizadores dessa defesa. O tom é de aventura leve, humor e engenhosidade: madeira, papelão, molas e potes viram equipamentos.

O jogador deve entender três coisas ao ver uma captura de tela: por onde os inimigos caminham, o que está protegendo e onde pode construir. O primeiro minuto deve terminar com uma torre disparando, moedas aumentando e uma decisão simples de melhoria. A coleção e a loja entram depois dessa primeira experiência.

Necessidades de um jogador de Roblox

Necessidade	Resposta do jogo	Verificação
Entender rapidamente	Primeiro inimigo e primeiro disparo em até 45 s como meta	Observar novos jogadores sem explicar
Jogar com amigos	Party no mesmo servidor, convite nativo quando permitido e revanche	Grupo de quatro conclui uma partida
Mostrar identidade	Skin, retrato, título e estandarte visíveis ao grupo	Reconhecer quem colocou cada torre
Sentir evolução	Compra de unidades por preço fixo e domínio cósmico	Jogador identifica próximo objetivo
Perceber contribuição	Dano, controle e apoio no resultado	Apoio não aparece como jogador inútil
Não se perder no celular	Botões grandes, confirmação e zoom de construção	Colocar cinco torres sem erro recorrente
Ter motivo para voltar	Novo mapa, especializações, desafios semanais futuros	Intenção espontânea de outra partida

Estas necessidades são hipóteses de design informadas pelas referências, e não resultados de entrevistas já realizadas.

Ciclo de jogo e telas

A sessão segue: entrar, escolher modo, montar equipe, defender, receber resultado e decidir a próxima partida. A campanha se divide em Jardim de Papel, Oficina de Lata e Sótão das Estrelas. Cada vitória normal abre o mapa seguinte. O modo Desafio de um mapa abre após vencer seu modo Normal.

O menu inicial é um painel 2D com Jogar em destaque, Coleção, Treino e Configurações. A loja é secundária. Retratos dos participantes e estado de prontidão ficam visíveis. Não há lobby 3D obrigatório, deslocamento até portais ou fila que impeça começar sozinho.

Cada jogador equipa até quatro torres diferentes; os quatro espaços existem desde o início. A conta nova possui Dardo, Pipoca e Goma. A quarta vaga pode ficar vazia. O menu sugere Lupa como próxima compra, sem obrigar essa ordem. A equipe fica travada quando a partida começa. Skins não alteram os atributos.

Primeiro contato e tutorial

O tutorial solo usa uma versão curta do Jardim, três ondas e as três torres iniciais. Dinheiro inicial de 1.200 sucatas, sem persistência do saldo. Onda 1: seis Fiapos; onda 2: seis Fiapos e quatro Coriscos; onda 3: duas Bolotas e oito Fiapos. Usa os mesmos atributos do Normal e os mesmos serviços de combate.

1. Mostrar caminho e farol; pedir para posicionar Dardo na célula (3,3).
2. Iniciar a primeira onda após confirmação; realçar dano, vida da base e sucata.
3. Após a primeira onda, guiar Goma para (3,4), formando um Circuito com Dardo. Explicar o bônus em uma frase.
4. Pedir uma melhoria de Dardo para L1 e mostrar o antes e depois. Oferecer Pipoca sem obrigar sua colocação.
5. Concluir a terceira onda e conceder 180 botões, uma única vez, quando o resultado for salvo.

Em falha, reiniciar a onda do tutorial a partir de um checkpoint em memória. Permitir pular explicações; pular não concede a recompensa, mas mantém as três unidades iniciais. O tutorial pode ser reaberto pelo menu. Nada de popup de compra durante essa sequência.

Treino

Treino solo libera temporariamente as seis torres, todas as especializações e sucata infinita. O jogador pode escolher inimigo, gerar 1 ou 10, limpar o tabuleiro e ligar indicadores de alcance e Circuitos. Não concede moeda, experiência, domínio, conquistas ou compras. Facilita testar uma torre antes de desbloqueá-la e serve como ferramenta de QA.

Regras da partida

O tabuleiro tem 16 colunas e 10 linhas, com células de tamanho lógico 1. O caminho é fixo. Uma torre ocupa uma célula fora do caminho e fora das decorações bloqueadoras. Alcance é circular e não há bloqueio de linha de visão. Todos os inimigos são terrestres; nenhuma unidade precisa de detecção oculta ou antiaérea nesta versão.

A base começa com 100 de vida. A onda só termina quando todos os seus grupos nasceram e não existe inimigo vivo. O grupo perde quando a base chega a zero e vence ao eliminar todos os inimigos da onda 20, incluindo o chefe. Se houver derrota e morte do último inimigo no mesmo passo da simulação, derrota tem precedência.

Há 15 segundos de preparação antes da primeira onda e 8 entre ondas. O botão Pronto antecipa o fim da preparação quando todos os participantes ativos concordam. Não antecipa ondas com inimigos vivos. A velocidade é 1x; 2x é gratuita e precisa de unanimidade no grupo. Um jogador pode solicitar retorno imediato a 1x. Tutorial fica em 1x.

Torres atacam automaticamente. O jogador pode colocar, melhorar, selecionar alvo e vender durante a onda. Venda devolve 70% da sucata investida, arredondada para baixo. Não há movimentação gratuita após confirmar a compra. Antes da confirmação, o fantasma de posicionamento pode ser movido livremente. Vender ou comprar nunca exige arrastar com precisão.

Limites de construção

Jogadores na largada	Torres por pessoa	Total máximo
1	18	18
2	12	24
3	10	30
4	8	32

O limite por espécie no catálogo também se aplica. O número de participantes, os multiplicadores e os limites são congelados na largada. Em uma saída, as torres continuam operando; não reduzir instantaneamente a vida dos inimigos. Regras de saída e retorno estão na especificação técnica.

Torres e escolhas de especialização

Torre	Personagem	Função	Especialização A	Especialização B
Dardo	Raposa de madeira	Ataque barato e constante	Rajada aumenta cadência	Fura lata ignora armadura
Pipoca	Tartaruga de corda	Explosão contra grupos	Festival amplia área	Demolidora enfrenta armadura
Lupa	Coruja de papelão	Longo alcance e elites	Olho de águia causa bônus em chefes	Observadora marca um alvo
Goma	Criatura em pote de cola	Reduz velocidade	Poça afeta um grupo	Supercola aplica lentidão maior
Voltz	Robô de mola	Corrente entre inimigos	Tempestade alcança mais alvos	Alta tensão concentra dano
Maestro	Caranguejo de rádio	Melhora cadência ao redor	Orquestra amplia alcance	Solo aumenta o bônus

Cada torre tem L0, L1, L2 e uma escolha exclusiva L3A ou L3B. Custos são pagos a cada avanço; não se pode ir diretamente de L0 para L3. A escolha final não muda até vender a torre. Não existe melhoria permanente de dano por nível de conta. Novos personagens ampliam opções, e domínio concede identidade visual.

Os números completos estão no catálogo e no JSON. São parâmetros iniciais de design, não balanceamento comprovado. A tabela inclui custo, dano, intervalo, alcance e efeitos para todos os 30 estados.

Circuitos por adjacência

Circuitos são o diferencial central e devem aparecer até o terceiro minuto do tutorial. Um par compatível em células ortogonalmente vizinhas se conecta por um fio discreto no chão. Diagonal não conta. Cada torre participa de no máximo um Circuito. A mesma regra funciona entre torres de jogadores diferentes, incentivando cooperação.

Par	Nome	Efeito
Dardo e Goma	Pega e acerta	Dardo causa 15% mais dano a qualquer inimigo com lentidão ativa
Pipoca e Lupa	Mira explosiva	Pipoca recebe mais 0,2 célula de raio de explosão
Voltz e Maestro	Ritmo elétrico	Queda de dano por salto melhora em 0,10, limitada a 0,90

Se houver vários vizinhos compatíveis, o servidor ordena os pares pela menor identificação de torre e depois pela maior, conectando o primeiro par livre. Identificações crescem conforme a construção é confirmada.

Não reordenar por dano, preço ou dono. A prévia deve mostrar exatamente qual par será ativado e que conexão poderá mudar após uma venda.

O efeito existe apenas enquanto as duas torres permanecem colocadas. Recalcular ao comprar e vender. Maestro mantém sua aura independentemente do Circuito. Desligar visual de fios nas configurações não desliga os efeitos; ao selecionar uma torre, mostrar sempre seu parceiro e o bônus numérico.

Inimigos e contrajogo

Fiapo ensina o básico. Corrisco exige cobertura de caminho. Bolota permite perceber dano constante. Latinha reduz dano físico e estimula uso de energia. Névoa reduz parte do dano de energia e favorece ataques físicos. Remendo cura aliados próximos e pode ser priorizado. Casulo deixa três Fiapos pequenos ao morrer. Brutamontes exige investimento em dano contra alvo único.

As resistências reduzem dano e nunca tornam um inimigo totalmente invulnerável. Exibir a característica no aviso da onda e no detalhe do inimigo. Não exigir que o jogador consulte uma wiki para compreender uma derrota.

Chefes

Chefe	Mapa	Comportamento	Resposta do jogador
Aspirador Rabugento	Jardim	Anuncia área de poeira e reduz cadência de torres nela por 4 s	Distribuir o investimento e usar o poder da base
Rei Ferrugem	Oficina	Alterna períodos de armadura maior	Preparar dano de energia ou Fura lata
Breu debaixo da cama	Sótão	Invoca quatro Corriscos à frente de seu progresso	Guardar cobertura e explosão para os reforços

Todos mostram barra de vida, nome, ícone da habilidade e anúncio visual antes da ação. Não há dano súbito sem aviso, efeitos que escondem o caminho ou compra de ressurreição na tela de derrota.

Poder compartilhado do farol

Pulso de Luz causa 180 de dano de energia a cada inimigo vivo e aplica 30% de lentidão por 3 segundos. Tem 60 segundos de recarga em tempo de simulação e começa carregado. Dano recebe as resistências normais e o multiplicador cooperativo de vida; não recebe bônus de torres. Chefes sofrem lentidão reduzida pela regra global. Todos podem ativar, e a primeira solicitação válida consome a recarga para o grupo. Prévia explica o efeito; um segundo toque confirma. O poder existe para oferecer uma decisão de emergência sem exigir controle de um herói.

Economia e progressão

Sucata é exclusiva da partida: começa em 650 por pessoa, compra e melhora torres, e desaparece ao encerrar. Botões persistem na conta: desbloqueiam personagens e itens cosméticos obtidos jogando. Não existe conversão de Robux para sucata ou botões na versão 1. Nenhum terceiro saldo é necessário.

Cada inimigo morto gera a recompensa indicada no catálogo para a equipe inteira, dividida em partes iguais entre os participantes da largada. O resto inteiro vai, um por vez, para jogadores em ordem crescente de UserId, com um cursor que gira entre mortes. A carteira do jogador ausente permanece em memória. Não há prêmio por último acerto, por dano individual ou por matar criaturas invocadas.

Concluir uma onda dá $120 + 15 \times \text{número da onda}$ de sucata a cada participante. Vencer concede 120 botões no Normal e 160 no Desafio. Perder concede $4 \times \text{ondas concluídas}$, limitado a 60. Sair antes do resultado não concede recompensa de conclusão. Tutorial dá 180 uma única vez. Treino dá zero.

Lupa custa 240 botões, Maestro 450 e Voltz 600. A primeira vitória normal, somada ao tutorial, já permite comprar Lupa. As três compras somam 1.290 botões: dez vitórias normais mais o tutorial rendem 1.380. Essa conta dimensiona o esforço nominal; duração real, derrotas e preferência de compra dependem dos testes.

Cada torre equipada recebe 10 pontos de domínio ao vencer, desde que tenha sido colocada ou prestado apoio por pelo menos 30 segundos. Derrota após concluir cinco ondas rende 4 pontos sob a mesma

condição. Marcos de 30, 100 e 250 pontos entregam adesivo, título e uma variação de skin, respectivamente. Domínio não muda atributos nem Circuitos. Não limitar pontos diários.

Cooperação e dificuldade

Vida dos inimigos comuns = vida do catálogo × multiplicador do mapa × multiplicador da dificuldade × [1 + 0,7 × (jogadores na largada - 1)]. Vida dos chefes usa sua própria vida de catálogo e os dois últimos fatores, sem aplicar novamente o multiplicador do mapa.

Normal usa fator 1. Desafio usa fator de vida 1,45 e velocidade 1,10. Não alterar quantidade, sucata ou resistências no Desafio. Arredondar vida uma única vez, para cima. A renda e o limite de torres devem ser testados em solo e em quatro pessoas; não assumir equilíbrio apenas porque a fórmula escala.

Interface e acessibilidade

No combate, a faixa superior mostra vida do farol, onda, sucata pessoal e participantes. O tabuleiro fica centralizado. A faixa inferior apresenta as quatro torres da equipe, Pulso de Luz e velocidade. O detalhe de torre abre em painel lateral no computador e em folha recolhível no celular, sem cobrir permanentemente o caminho.

Posicionamento tem três passos: tocar a unidade, tocar a célula e confirmar o preço. Mostrar fantasma, alcance, validade e Circuito previsto. Célula inválida tem ícone e texto, além de cor. Se a célula ocupar menos de 36 pixels físicos de interface, ativar ampliação da região ao escolher a unidade; botões continuam com pelo menos 44 × 44 pixels de interface, meta de 48. Permitir arrastar o mapa com dois dedos e voltar à visão completa com um botão.

Mostrar informações da próxima onda com ícones e frase curta. O resultado informa ondas superadas, recompensa, progresso de domínio e contribuição por dano, tempo de lentidão e dano adicional produzido por apoio. Esses dados são informativos; não mudam o pagamento. Oferecer Tentar novamente e Voltar ao menu, mantendo a party quando possível.

Incluir volume separado para música e efeitos, redução de movimento, partículas reduzidas, números de dano opcionais e padrão visual alternativo para Circuitos. Usar formas e ícones para equipe, resistência e estado. Preservar menus e controles nativos do Roblox. Console e VR não entram na validação inicial nem devem ser anunciados como suportados.

Monetização e retorno

A versão inicial pode ser lançada gratuitamente com a loja desligada. Após validar diversão e salvar dados com segurança, ativar um passe permanente Pacote Fundador: seis skins visuais em paleta de oficina, um estandarte e um título. Preço inicial de teste: 149 Robux, definido no painel da plataforma e consultado pela interface. É hipótese de produto, não estimativa de receita. Conferir preço e disponibilidade reais antes de mostrar a compra.

O passe não concede dano, dinheiro de partida, aceleração exclusiva, vagas extras, desconto em torres ou vantagem de domínio. Não incluir recompensas aleatórias pagas, anúncios obrigatórios, troca de itens, caixas ou revenda. A decisão reduz o trabalho de economia e deixa o mérito da estratégia observável.

O retorno no lançamento vem da campanha, do Desafio e do domínio dos seis personagens. Após observar jogadores, avaliar desafios semanais com equipe fixa, novo mapa e eventos cosméticos. Não prometer uma atualização por semana sem capacidade de produção. Toda adição deve preservar a utilidade das torres já existentes.

Escopo de produção e lançamento

Obrigatório para a versão 1

Tutorial, treino, coleção, equipe, três mapas, duas dificuldades, vinte ondas por mapa com parâmetros definidos, seis torres completas, três Circuitos, oito inimigos comuns, três chefes, Pulso de Luz, modo solo, cooperação, persistência, configurações, localização, resultado e revanche. A infraestrutura de loja deve existir com desligamento seguro; ativá-la é uma decisão separada de publicação.

Fora da versão 1

PvP, mercado entre jogadores, personagens licenciados, gacha, clãs, passe de temporada, campanha narrativa longa, labirinto livre, torre de renda, herói móvel, múltiplos caminhos simultâneos, modo infinito com economia própria, reconexão garantida entre servidores e suporte a console. Essas ideias não devem aparecer como botões inativos no produto.

Marcos de trabalho

Marco	Entrega	Condição para avançar
M0 Fundação	Projeto abre no Studio e renderiza tabuleiro 2D	Entrada, resolução e coordenadas corretas
M1 Combate	Jardim, Dardo, Goma, inimigos e três ondas	Uma partida curta pode ser vencida e perdida
M2 Diferencial	Seis torres, especializações, Circuitos e treino	Combinações verificadas sem ambiguidade
M3 Conteúdo	Três mapas, vinte ondas, chefes e Desafio	Todas as condições de vitória e derrota funcionam
M4 Serviço	Cooperação, party, perfis e recompensas	Concorrência e falhas de salvamento testadas
M5 Apresentação	Arte, áudio, tutorial e interface final	Leitura no celular e testes com jogadores
M6 Candidato	Desempenho, telemetria e publicação de teste	Critérios técnicos e de produto atendidos

Não existe estimativa responsável de prazo completo sem conhecer a equipe, a qualidade dos assets e a capacidade de teste. O Claude Code pode implementar scripts, estrutura e ferramentas; produção visual final, uploads, configurações de conta e avaliação de diversão exigem recursos e validação próprios. Os marcos são uma ordem de construção, não uma promessa de geração integral em uma execução.

Validação de diversão e métricas

Fazer primeiro uma rodada observada com 8 a 12 pessoas, parte iniciante em TD e parte habituada ao Roblox, incluindo uso real de celular. Essa amostra serve para localizar problemas, não para provar retenção. Pedir que joguem sem explicação externa e observar onde deixam de entender a interface.

Critérios iniciais propostos: pelo menos 8 de 10 participantes conseguem colocar e melhorar uma torre; pelo menos 7 explicam um Circuito depois do tutorial; pelo menos 6 escolhem espontaneamente uma segunda partida. Se não ocorrer, revisar tutorial, duração e feedback antes de ampliar conteúdo ou comprar tráfego.

Instrumentar entrada, início do tutorial, primeira torre, primeira melhoria, primeiro Circuito, fim do tutorial, início da partida, onda atingida, fim, recompensa salva e início da segunda partida. Registrar build, mapa, dificuldade, tamanho do grupo e dispositivo. O servidor emite resultados; pedidos do cliente não são prova de vitória.

Em beta, acompanhar taxa de conclusão do tutorial, tempo até primeiro disparo, abandono por onda, duração, vitórias por mapa e grupo, seleção de torres, segunda partida e retorno D1 e D7. D1 e D7 são retorno no dia de calendário UTC correspondente após a primeira sessão, com denominador apenas de coortes já maduras. Separar celular de computador. Metas de retenção comercial precisam ser fixadas após o primeiro baseline; não apresentamos benchmarks inventados.

Riscos e decisões de resposta

O 2D pode perder parte da expressão do avatar típica do Roblox. Compensar com retratos, estandartes, torres do jogador e presença da party; medir se o grupo se reconhece. A adjacência pode parecer difícil no celular: priorizar prévia e ampliação contextual. Um elenco pequeno pode parecer pouco colecionável: investir em duas especializações e domínio antes de multiplicar personagens. Nenhuma dessas hipóteses substitui teste com jogadores.

2 Especificação técnica

Contrato de implementação

Implementar Quintal em Guarda em Luau com tipagem estrita, Roblox Studio e projeto Rojo. O gameplay deve ser 2D real: dados de posição em duas dimensões e apresentação por objetos de interface. Não substituir por um mundo 3D com câmera distante. A arte conceitual usa volume desenhado para comunicar materiais; não exige modelos 3D.

O JSON em dados/balanceamento_v1.json é a fonte canônica dos parâmetros numéricos. Este documento define a semântica desses campos e os contratos de sistemas. O game design define o comportamento do produto; o documento de arte define sua apresentação. Se surgir conflito, registrar a divergência, corrigir os arquivos afetados e manter uma única versão, sem inventar silenciosamente uma regra.

Congelar versões das ferramentas no repositório no início da implementação. Usar APIs documentadas no Creator Hub e verificar os nomes atuais no ambiente de destino. As taxas de atualização e os limites abaixo são escolhas do projeto, não garantias da plataforma.

Arquitetura 2D

Usar um ScreenGui de aplicação, com telas de menu e combate, um Frame para o tabuleiro e ImageLabels para cenário, unidades e efeitos. ImageRectOffset e ImageRectSize permitem recortes de atlas. Uma moldura de interface não deve participar do cálculo de combate. [Contêineres de interface](#) e [ImageLabel](#).

Desativar CharacterAutoLoads no servidor e criar explicitamente o ScreenGui em PlayerGui no bootstrap do cliente. Não depender da cópia automática de StarterGui quando o avatar não é carregado. Manter um ambiente 3D vazio e estável atrás da aplicação, câmera Scriptable e painel de fundo opaco. Não criar Humanoids, Parts de inimigos, física, colisão ou PathfindingService para este tabuleiro.

Coordenadas lógicas são centros de células inteiras: x de 0 a 15 e y de 0 a 9. Uma célula ocupa $[x-0,5, x+0,5)$ por $[y-0,5, y+0,5)$. O tabuleiro desenhado mede 960 por 600 unidades visuais. No estado sem zoom, $px = left + (x + 0,5) \times largura/16$; $py = top + (y + 0,5) \times altura/10$. A transformação inversa arredonda pela célula correspondente e rejeita pontos fora dos limites.

BoardTransform centraliza escala, zoom e deslocamento; todo input passa por sua transformação inversa. Ela considera AbsolutePosition, AbsoluteSize e a área segura do ScreenGui. Não somar um inset duas vezes. Testar bordas, mudança de resolução e zoom. A proporção do mapa permanece 16:10. HUD é responsivo e não estica o tabuleiro.

Apresentação e seleção

Separar a lógica de seleção do sprite. Usar célula, entidade e distância ao ponto lógico para resolver sobreposição; priorizar a torre da célula tocada, depois o inimigo mais próximo dentro do raio de seleção. Tocar em área vazia fecha o detalhe. Entrada com mouse oferece 1 a 4 para escolher torre, Esc para cancelar e U para melhorar quando a escolha é única. Especialização e venda sempre mostram confirmação.

Sprites normalizados se ancoram nos pés, em (0,5; 0,82) de um quadro quadrado. Os PNGs originais entregues têm enquadramentos distintos; validar as âncoras sugeridas no manifesto e normalizar conforme o capítulo 8 antes de aplicar esse padrão. ZIndex do mundo usa faixas: chão 0 a 9, caminho 10 a 19, Circuitos 20 a 29, sombras 30 a 39, entidades $100 + \text{floor}(y \times 100)$ com pequeno desempate por tipo, efeitos 1200 a 1299, seleção 1300 a 1399. Menus usam outro DisplayOrder. Atualizar profundidade apenas quando necessário. Não usar uma imagem que contenha texto para substituir um botão acessível.

Organização dos arquivos

```
default.project.json
toolchain.toml
src/
  shared/
    Types.lua
    Config/Balance.lua
    Config/Maps.lua
    Config/Assets.lua
    Config/Products.lua
    Config/Localization.lua
    Math/Path.lua
    Math/Grid.lua
    Rules/Combat.lua
    Rules/Circuits.lua
    Rules/Rewards.lua
    Net/Protocol.lua
  server/
    Bootstrap.server.lua
    Services/MatchService.lua
    Services/WaveService.lua
    Services/TowerService.lua
    Services/EnemyService.lua
    Services/EconomyService.lua
    Services/PartyService.lua
    Services/ProfileService.lua
    Services/RewardService.lua
    Services/ShopService.lua
    Services/TelemetryService.lua
  client/
    Bootstrap.client.lua
    Controllers/ScreenController.lua
    Controllers/InputController.lua
    Controllers/MatchController.lua
    Controllers/AudioController.lua
    View/BoardRenderer.lua
    View/BoardTransform.lua
    View/SpritePool.lua
    View/Hud.lua
    View/Collection.lua
    View/Party.lua
    View/Settings.lua
  tests/
  assets/source/
  assets/export/
  tools/import_balance.py
  docs/SETUP.md
  docs/TEST_REPORT.md
  docs/MANUAL_ACTIONS.md
```

shared vai para ReplicatedStorage; server para ServerScriptService; client para StarterPlayerScripts. Os nomes são uma estrutura recomendada, e podem mudar mantendo separação de responsabilidades. Dados de perfil, recompensas e validação nunca ficam sob controle do cliente. Regras puras recebem dados e retornam resultados sem depender de interface ou serviços Roblox, permitindo teste determinístico.

Gerar Balance.lua a partir do JSON com validação e saída determinística; o jogo não deve baixar configuração da internet durante a partida. O build falha quando há ID desconhecido, custo inválido, salto de evolução inexistente, caminho diagonal, bloqueio sobre caminho ou frame fora do atlas. Manter relatório de versão e checksum dos dados usados.

Estado e relógio

MatchState contém matchId único, configVersion, participantes da largada, seed, modo, mapa, dificuldade, fase, waveIndex, baseHP, simTime, speedMultiplier, entidades, carteiras e recompensa. Fases válidas: Lobby, Loading, Preparation, Wave, Intermission, Won, Lost, RewardPending e Closed. Treino e tutorial são modos explícitos, não flags esquecidas em uma partida recompensada.

Um servidor público aceita até 12 pessoas e gerencia instâncias lógicas de partidas isoladas. Cada partida admite de uma a quatro. A implementação inicial não depende de teleportes ou fila global; party e seleção de grupo operam entre jogadores presentes no mesmo servidor. O menu permite solo imediato ou salas com lugares disponíveis. Uma pessoa pertence a apenas uma partida ativa.

Simulação no servidor em passos de 0,05 s, usando acumulador. O multiplicador 2x aumenta o tempo simulado, mantendo o tamanho do passo. Máximo de seis passos por Heartbeat; preservar o restante limitado a 0,5 s e registrar sobrecarga. Se o limite for repetidamente atingido, reduzir 2x para 1x com aviso. Não pular movimento, dano ou pagamentos para recuperar frames.

Todos os cooldowns de jogo usam simTime. Temporizadores de sessão, desconexão, rede e salvamento usam relógio real do servidor. Ao pausar a renderização de um cliente, a partida do grupo continua. O modo solo não ganha pausa global no lançamento.

Ordem por passo: processar comandos validados em ordem de chegada; criar inimigos agendados; mover; resolver chegadas à base; atualizar estados; resolver cura; atualizar chefes; adquirir alvos; aplicar ataques ordenados pelo ID da torre; processar mortes, divisões e pagamentos; verificar fim da onda e da partida. Se a base zerar na etapa de movimento, encerrar em derrota antes de executar novos ataques. Entidades mortas não voltam a atacar nem geram dois pagamentos.

Movimento e combate

Pré-calcular comprimentos dos segmentos e comprimento acumulado do caminho. Inimigo mantém distância escalar s, velocidade, vida e estados. Posição é a interpolação no segmento correspondente a s. Alvo First é a maior razão s/comprimento total; Last é a menor; Strongest é a maior vida atual; Closest é a menor distância euclidiana à torre. Empates usam menor ID. Todas as torres de dano permitem esses quatro modos; apoio não recebe seletor.

Adquirir um alvo somente se vivo, no mesmo match e dentro de alcance, incluindo igualdade. Acerto é instantâneo na simulação; projéteis desenhados são cosméticos. Assim, o ponto visual de um projétil não decide dano. A primeira aquisição permite disparo imediato. Um upgrade conserva a fração restante do cooldown, recalculada pelo novo intervalo, para evitar disparos grátis ao comprar. Sem alvo, o cooldown chega a zero e permanece pronto.

Fórmula de dano

```
bonus = (1 + bonusContraChefe) * (1 + bonusCircuito)
vulnerabilidade = 1 + maiorMarcaAtiva
armaduraEfetiva = clamp(armaduraAtual * (1 - armorIgnore), 0, 0.80)
resistencia = armaduraEfetiva para physical
resistencia = energyResistance para energy
danoFinal = danoBase * bonus * vulnerabilidade * (1 - resistencia)
```

Campos ausentes valem zero, exceto multiplicadores cuja base é 1. armorIgnore vale fração de armadura ignorada: 0,5 remove metade; 1 remove toda. Vida e dano interno usam valores decimais; arredondar apenas para a interface. A vida inicial é arredondada para cima uma vez. Resistência de energia padrão é zero. Bônus contra chefe só vale para entidades com isBoss.

Lupa Observadora aplica marca de 10% por 3 s depois de seu acerto; o próprio acerto inicial não recebe a nova marca. Marcas não se somam: vale a maior, renovando duração quando reaplicada. Cada efeito recebe sourceTowerId e expiresAt. Nunca executar um task.wait por efeito ou inimigo.

Área corrente e apoio

Explosão afeta o alvo primário e os inimigos mais próximos do seu centro, em raio inclusive, até maxTargets no total; dano é igual por alvo. Empates usam ID. O bônus Mira explosiva aumenta apenas splash, não alcance, dano ou limite de alvos.

chains é o número total de alvos, incluindo o primeiro. Cada salto procura o inimigo ainda não atingido mais próximo do anterior dentro de chainRadius, sem exigir alcance direto da torre. Dano do alvo de índice k, começando em zero, é $\text{damage} \times \text{chainFalloff}^k$. Recalcular alvo a cada disparo; uma corrente nunca atinge a mesma entidade duas vezes.

Lentidões não se somam; usar a maior ainda ativa, limitada a 50%. Em chefes, multiplicar a força por 0,35 antes do limite. Goma L3A aplica seu slow e dano a todos os alvos da explosão. Um alvo lento continua elegível para Pega e acerta independentemente da fonte da lentidão.

Aura do Maestro só afeta torres de dano; vale o maior haste em alcance, inclusive entre donos diferentes. Maestro não acelera a si mesmo nem outros Maestros. Intervalo efetivo = $\text{intervaloBase} / (1 + \text{maiorHaste})$. Poeira do Aspirador aplica depois / 0,75 durante a duração; áreas de poeira não se acumulam. Sem críticos, esquiva ou aleatoriedade de acerto na versão 1.

Remendo cura a cada três segundos de simulação, até a vida máxima, os demais inimigos comuns a até 1,5 célula, excluindo a si, chefes e outros Remendos. Cada alvo recebe no máximo um pulso de cura por passo, mesmo com vários curadores. Pulso é $12 \times$ fator de vida comum da partida. Casulo cria três Fiapos na própria posição ao morrer, com $hp = 12 \times$ fator de vida comum, velocidade 1,6, leak 1 e bounty 0. Se Casulo vaza na base, não se divide. Criaturas do Breu também têm bounty 0.

Chefes e ondas

Catálogo traz vinte listas ordenadas de grupos. Criar o primeiro membro imediatamente ao começar cada grupo; demais respeitam `spawnInterval`; `groupGap` ocorre entre o último spawn de um grupo e o primeiro do próximo. Após todos os grupos da onda 20, aguardar dois segundos e criar o chefe do mapa. Invocações do chefe contam para a conclusão da onda.

A vida dos chefes é fixa por mapa no catálogo, multiplicada apenas por cooperação e dificuldade. Slow afeta movimento, sem mudar cadência de habilidades. O relógio de cada habilidade começa no nascimento; ao chegar a `period`, iniciar o aviso de telegraph segundos e então executar. O próximo aviso começa `period` segundos após o anterior. Fases terminais cancelam eventos agendados.

Aspirador escolhe a torre com maior sucata investida, desempate por ID, e fixa um círculo de raio 2 no local durante o aviso. Ao resolver, torres naquele círculo sofrem poeira por 4 s, mesmo que o chefe se mova. Nenhuma torre é removida. Rei Ferrugem recebe mais 0,35 de armadura por 5 s após seu aviso, depois retorna ao valor base. Breu invoca quatro Coriscos em $s + 0,5$ célula, limitado ao comprimento do caminho menos 0,1; não invocar atrás da base.

O Pulso de Luz é resolvido no passo em que o comando é aceito. Seu dano base 180 é multiplicado por $[1 + 0,7 \times (N - 1)]$ e processado como energia para cada inimigo vivo. Não multiplicar pelo mapa ou pelo modo Desafio. A marca de vulnerabilidade do alvo se aplica; bônus de torres não. A lentidão segue as regras gerais. Disponibilidade, consumo e efeito são atômicos para impedir dois usos no mesmo passo.

Rede e autoridade

O cliente envia intenções. O servidor valida estado, dono, saldo, configuração, cooldown, coordenadas, tamanho e frequência. Vida, dano, recompensa e preço são calculados no servidor. Essa separação segue a orientação de segurança do Roblox. [Validação da fronteira cliente servidor](#).

Um `RemoteEvent Command` recebe `{protocolVersion, sessionSequence, requestId, action, payload}`. `requestId` é uma string de até 40 caracteres; `sessionSequence` é um inteiro crescente por sessão de cliente. Rejeitar repetição já processada por usuário e match, devolvendo a resposta anterior sem repetir gasto. Guardar 128 respostas recentes e a maior sequência aceita; comandos antigos fora da janela são rejeitados. A reconexão inicia uma nova sessão autenticada pelo servidor. Campos desconhecidos, tabelas profundas, strings excessivas, valores NaN ou infinitos são recusados antes de qualquer operação custosa.

Ação	Payload permitido	Validação central
PlaceTower	matchId, towerId, x, y	Equipada, desbloqueada, célula inteira válida, saldo e limites
UpgradeTower	matchId, entityId, nextState	Dono, transição legal e saldo
SellTower	matchId, entityId	Dono e entidade ainda ativa
SetTarget	matchId, entityId, mode	Dono e enum válido
CastPulse	matchId	Participante ativo, fase de combate e recarga
SetReady	matchId, ready	Preparação ou intervalo
VoteSpeed	matchId, value	Valor 1 ou 2, voto atual do participante
SetLoadout	towerIds	Menu, até quatro IDs distintos desbloqueados
BuyTower	towerId	Menu, preço do catálogo, saldo e ainda não possuída
StartMatch	partyId, mapId, mode	Líder, presentes prontos, mapa liberado para todos

Respostas incluem requestId, ok, errorCode, balance e revision. Erros previstos: INVALID_CELL, OCCUPIED, NOT_OWNER, INSUFFICIENT_FUNDS, LIMIT_REACHED, INVALID_STATE, PROFILE_UNAVAILABLE, RATE_LIMIT e STALE_REQUEST. A interface traduz códigos em mensagens úteis e remove estado pendente. Nada de entregar ao cliente a responsabilidade de desfazer um gasto confirmado.

Aplicar limite inicial de dez comandos de combate por segundo por usuário com rajada de vinte. Compras persistentes: uma por segundo, rajada de duas. Pedidos de snapshot completo: um a cada dois segundos. Valores são orçamento do projeto e precisam ser medidos. Não banir automaticamente por um erro de rede; registrar violações repetidas e descartar pedidos inválidos.

Replicação: estado inicial completo, mudanças a 10 Hz com revision crescente e snapshot de reconciliação a cada 2 s. Agrupar spawn, death, dano, torres, carteira e fase. Eventos cosméticos podem ser agregados. Cliente mantém buffer de 100 ms para interpolar posição, ignorando revisões antigas. Morte confirmada remove a entidade sem esperar projétil. Solicitação de recuperação é validada e o snapshot nunca inclui perfil de outro jogador.

Enviar somente dados da partida do destinatário. Não usar FireAllClients para expor todas as salas. Não transmitir posição individual de cada inimigo a cada frame se distância s, velocidade e timestamp já permitem apresentação. Seleção, alcance desenhado, câmera e animação são locais.

Party e ciclo de sessão

Um servidor mantém parties de até quatro pessoas, com líder, membros, prontidão e sala. Convites internos usam lista de presentes; convite nativo de amigos só aparece quando a API informar disponibilidade. Entrar pelo Roblox não garante vaga no servidor cheio; mostrar estado real e permitir solo. Não prometer matchmaking entre servidores nesta versão.

O líder escolhe mapa e dificuldade que todos possuem. Trocar a composição limpa a prontidão. Uma saída no menu transfere liderança ao membro mais antigo. Ao começar, o servidor cria matchId e congela participantes. Quem chega durante a partida aguarda no menu e pode entrar na próxima. Sem recompensas para espectadores.

Ao desconectar, manter carteira e torres até o término; não permitir que outro jogador venda ou gaste por esse dono. Um retorno ao mesmo servidor e match restaura o controle por UserId e snapshot. Isso é retomada limitada, sem garantia de roteamento Roblox ao mesmo servidor. Depois de 30 s ausente, a pessoa deixa de contar para unanimidade de Pronto e 2x. Os fatores de combate permanecem os da largada.

Se não houver nenhum participante conectado por 60 s, encerrar como Abandoned, sem recompensa de conclusão. Quem estiver ausente no instante do resultado não recebe o prêmio; informar essa regra no diálogo de saída. Quem participou e está conectado recebe o mesmo resultado, sem exigência de último acerto. Sessões sem nenhuma ação e sem torre colocada durante toda a partida não recebem domínio, mas recebem os botões de participação; acompanhar abuso antes de adicionar regras punitivas.

No resultado, o clique de revanche marca prontidão. Após todos os presentes confirmarem, iniciar nova partida com novo ID. Ausentes saem da party de revanche após 30 s. O líder pode voltar ao menu sem transportar outros jogadores contra sua escolha.

Persistência e recompensa

Perfil persistente por UserId: schemaVersion, buttons, ownedTowerIds, loadout, cosmeticsOwned, cosmeticLoadout, masteryByTower, mapUnlocks, tutorialRewardClaimed, settings, lifetimeStats, recentMatchRewards e sessionLock. Valores monetários são inteiros não negativos; validar catálogo e migração antes de aceitar perfil.

Usar DataStoreService apenas no servidor, com UpdateAsync para alterações concorrentes, pcall e repetição com espera progressiva. A documentação alerta que o Studio pode acessar os mesmos dados de produção; o ambiente de teste deve ter outro universo ou namespace. [Data stores](#).

Adquirir lock de sessão por UpdateAsync contendo JobId, token e expiresAt de 120 s; renovar a cada 30 s. Toda gravação compara o token. Se perder o lock, suspender novas mutações persistentes e compras, mantendo a interface com mensagem clara. Quando outro servidor possui lock válido, permitir tentar novamente ou entrar em treino sem progresso; nunca substituir por perfil vazio.

Salvar alterações sujas a cada 60 s com jitter, em desbloqueios, resultado, saída e BindToClose. Serializar gravações por usuário e verificar orçamento disponível. Callback de UpdateAsync não pode ceder execução nem gerar efeitos externos. Falha de carregamento nunca autoriza gravar defaults sobre um perfil existente. Migrações v1 para v2 etc. são puras, idempotentes e cobertas por teste com cópias.

Resultado deve criar um registro durável MatchResult com matchId, versão, resultado, participantes elegíveis e valores calculados. A fase RewardPending só exibe “salvo” depois de confirmar persistência. Cada perfil aplica o resultado junto com o registro matchId na mesma alteração atômica, sem dar recompensa duas vezes. Se aplicar a alguns perfis e falhar em outros, o registro permite repetir somente os pendentes.

Guardar IDs de recompensa por 30 dias, com limite de 1.024 entradas por perfil; bloquear reprocessamento de resultados mais antigos que 30 dias antes da consulta à lista. Ao atingir o limite dentro da janela, preservar a proteção e registrar necessidade de migração para ledger particionado, sem descartar IDs ainda válidos. Manter referências de resultados pendentes no perfil antes de iniciar a partida, para consulta na próxima sessão. Remover referência apenas após liquidar ou registrar abandono.

Se o servidor cair antes de criar um resultado durável, uma partida ainda em andamento não é recuperada nesta versão. Não prometer recompensa que não foi registrada. Em falha no final, mostrar “recompensa pendente” e reconciliar na próxima entrada. O tutorial segue o mesmo princípio, com chave única tutorial-v1 por conta.

Loja e configuração externa

O único produto da versão 1 é um passe permanente. Products.luau contém founderPassId opcional e shopEnabled falso por padrão. Sem ID válido ou consulta de preço bem-sucedida, ocultar o botão de compra e manter o restante jogável. Validar posse do passe no servidor, consultar preço pela API e tratar cancelamento sem conceder itens. [Passes no Creator Hub](#).

Não implementar developer products apenas por hábito. Se forem autorizados em uma expansão, conceder itens com ProcessReceipt e ledger idempotente por PurchaseId, nunca pelo evento que apenas indica fechamento do popup. [Developer products](#).

Todos os assets e IDs de conta ficam em manifesto. Cada entrada especifica id lógico, ContentId, arquivo de origem, autor, direito de uso, dimensões e status de moderação. Arquivo local PNG não funciona automaticamente como ContentId Roblox. Sem upload aprovado, usar formas de interface provisórias claramente identificadas no modo de desenvolvimento; não alegar que a arte final está integrada.

Orçamentos e desempenho

Item	Meta inicial para medir
Inimigos vivos por partida	Até 180 no conteúdo normal; estresse com 250
Torres por partida	Até 32
Servidor com 12 jogadores	Validar três partidas de quatro e doze partidas solo
Renderização	60 fps em computador de referência e 30 fps estáveis no celular de referência
Passo de simulação	p95 abaixo de 8 ms no cenário de carga definido
Tráfego por cliente em combate	Meta média abaixo de 35 KB/s, medir no profiler
Objetos visuais simultâneos	Meta abaixo de 1.800 GuiObjects por cliente
Texturas decodificadas próprias	Meta abaixo de 96 MiB residentes por partida
Áudio simultâneo	Até 16 vozes, com prioridade para avisos

Esses valores são critérios de projeto a validar em dispositivos nomeados no relatório. Não são limites oficiais nem resultados obtidos. Se um spawn superar 250 inimigos, adiar spawns normais; uma invocação aguarda espaço, sem desaparecer nem render moeda. O roteiro normal deve ser ajustado para não depender desse mecanismo.

Usar pools de sprites e efeitos; não criar e destruir toda a interface por onda. Um único loop visual atualiza os objetos ativos. Para alvos, começar com busca simples medida e migrar a índice espacial por células se o

profiler mostrar necessidade. Nunca escolher arquitetura mais complexa sem evidência. Carregar assets do mapa e equipe primeiro; carregar menu e demais skins sob demanda.

Telemetria e depuração

Logar eventos com versão, matchId, modo, mapa, onda, duração, grupo e categoria de dispositivo, sem texto livre de chat. O ID do evento de resultado é estável para deduplicação. Funis podem ser enviados ao AnalyticsService conforme a documentação vigente; falha de telemetria não pode interromper combate ou salvamento. [Eventos de funil](#).

Ferramentas de desenvolvimento: iniciar em uma onda, conceder sucata de teste, mostrar caminhos, inspecionar alvos, alternar profundidade de desenho e exportar resumo da partida. Autorizar somente em Studio ou por allowlist de UserIds no servidor de teste. Nenhum comando administrativo deve confiar em um botão escondido no cliente. Não publicar remotes de depuração abertos.

Entregáveis exigidos do Claude Code

Código organizado, projeto Rojo, instruções de instalação, geração dos dados, testes de regras, build de lugar quando as ferramentas estiverem disponíveis, manifesto de assets, relatório de verificações e lista objetiva de passos manuais. Não incluir segredos ou credenciais. Não automatizar publicação em conta sem autorização de quem a controla.

Rojo permite trabalhar nos arquivos e sincronizá-los com Studio, mas não substitui o teste no motor. Validar combate solo, simulação cliente servidor e vários clientes. [Documentação Rojo](#) e [Modos de teste do Studio](#).

Entrega “código pronto” significa que os módulos e testes disponíveis foram concluídos. Entrega “jogo validado” exige execução no Roblox Studio e em clientes reais. Entrega “publicado” exige um lugar publicado, permissões e assets aprovados. O relatório deve distinguir esses estados com precisão.

3 Direção de arte

Objetivo visual

Criar um pequeno mundo de brinquedos com aspecto artesanal, formas grandes e personagens expressivos. A arte precisa comunicar função antes de detalhe: uma raposa com disparador é ataque rápido, uma tartaruga com tubo largo é explosão, uma coruja com luneta é alcance. O cenário deve convidar a jogar e continuar legível depois que houver dezenas de inimigos na tela.

Esta direção acompanha o game design e a especificação técnica. A prancha conceitual foi gerada com a ferramenta de imagens integrada ao Codex. Ela é referência de personagem, cor e material; não é screenshot de um jogo existente, arquivo de sprites, mapa implementado ou garantia de consistência quadro a quadro. O prompt utilizado está em arte/PROMPT_CONCEITO.txt.



Prancha conceitual original dos seis personagens e do quintal

O layout de produção usa a grade e os caminhos ortogonais do catálogo. Não copiar literalmente as curvas, os pedestais ou a perspectiva ilustrada da prancha para a geometria de jogo. A especificação do tabuleiro tem precedência; preservar o caráter acolhedor, os materiais de brinquedo e as silhuetas.

Na versão 1.1, o pacote inclui 40 PNGs individuais na pasta `Assets_Quintal_em_Guarda_v1`: 30 estados de torres, três terrenos e sete imagens de cenário e base. Eles são poses estáticas utilizáveis na implementação. O capítulo de integração explica caminhos, upload, escala, movimentos procedurais e os quadros de animação ainda pendentes.

Linguagem visual

Ilustração 2D cartoon com contorno azul muito escuro, cores chapadas e até duas faixas de sombra. Cenários usam textura suave de papel; personagens e inimigos devem manter áreas de cor limpas. O volume é desenhado, sem iluminação 3D dinâmica. O ambiente usa visão superior; os personagens são sprites com rosto visível e pés ancorados no tabuleiro, como peças vistas de cima em três quartos.

As formas aliadas são arredondadas, estáveis e montadas com peças reconhecíveis. Os inimigos são assimétricos, macios e escuros, com olhos claros. Os chefes ampliam uma ideia simples, como um

aspirador que virou criatura, em vez de acumular muitos acessórios. Violência é de brinquedo: desmontar, soltar poeira e saltar; sem sangue ou sofrimento realista.

Usar uma fonte de luz desenhada no alto à esquerda e sombra elíptica curta separada do sprite. A sombra fica debaixo dos pés, não dentro da textura do personagem, para evitar duplicação. Contornos externos: 3 a 4 px no export de 128 px; detalhes internos: 1 a 2 px, eliminados quando atrapalharem leitura.

O que aprovar e o que recusar

Aprovar	Recusar
Silhueta distinguível em 32 px	Seis personagens com o mesmo corpo e cores diferentes
Arma ou acessório que explica a função	Muitos enfeites menores que um pixel na tela
Upgrade altera forma e material	Upgrade que só troca número ou brilho
Cenário suave com caminho evidente	Textura tão contrastada quanto os inimigos
Efeito curto com ponto de impacto visível	Explosão que cobre a rota inteira
Pele cosmética mantém função e escala	Skin que parece outra torre ou oculta status

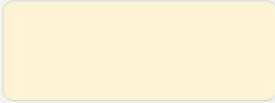
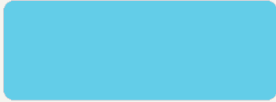
Paleta e semântica

Token	Cor	Aplicação
ink	#19283F	Contornos, texto escuro e foco
cream	#FFF3D6	Painéis e fundo claro
teal	#2FB7A0	Ambiente vivo e ações positivas
gold	#FFC857	Sucata, melhoria e energia do farol
coral	#FF775E	Ênfase e personalidade dos brinquedos
enemy	#7D63B8	Família de poeira e Breu
cyan	#63CDE8	Energia, alcance e corrente
danger	#B63E48	Alerta com ícone e texto
muted	#66768A	Informação secundária sobre fundo claro

Usar ink sobre cream para texto principal. Não usar texto branco pequeno sobre gold, cyan ou teal. Informações de sucesso, erro e dono incluem ícones, padrões ou rótulos além da cor. Medir contraste do produto final: meta de 4,5:1 para texto normal e 3:1 para texto grande. Esses são critérios de QA visual do projeto.

As cores de donos usam círculo, quadrado, triângulo e losango nos marcadores, além de quatro tonalidades. Resistência física usa escudo de lata; resistência de energia usa espiral; cura usa remendo; lentidão usa gota com seta. Não reaproveitar o mesmo símbolo para moeda e energia.

Paleta e leitura visual

 Ink #19283F / Texto e contorno	 Cream #FFF3D6 / Painéis	 Teal #2FB7A0 / Ambiente	 Gold #FFC857 / Sucata
 Coral #FF775E / Brinquedos	 Enemy #7D63B8 / Poeira	 Cyan #63CDE8 / Energia	 Danger #B63E48 / Alertas

Cor reforça a informação. Forma, ícone e texto carregam o significado.

Paleta e símbolos de função

Personagens aliados

Dardo

Raposa de madeira laranja, orelhas triangulares, cauda larga e disparador de elástico. Corpo compacto e atitude curiosa. L0 tem disparador simples; L1 ganha braçadeira; L2 tem mecanismo duplo. Rajada acrescenta tambor pequeno e pose inclinada. Fura lata usa ponta metálica maior e mira curta. O rosto e a cauda continuam reconhecíveis em todas as skins.

Pipoca

Tartaruga de corda com casco circular e tubo largo nas costas. Amarelo e verde escuro. L0 tem tubo curto; L1 reforço no casco; L2 acrescenta aro e manivela. Festival abre um bocal mais largo e pequenas bandeiras. Demolidora ganha tampa metálica e tubo mais comprido, sem parecer um tanque realista. A animação contrai o casco antes do disparo.

Lupa

Coruja alta de papelão com olhos grandes e luneta, usando tons de papel, âmbar e marrom. L1 acrescenta suporte; L2 amplia a luneta. Olho de águia alonga a lente e adiciona uma pena triangular. Observadora usa lente dupla e pequeno marcador luminoso. Silhueta vertical, comparada à largura de Pipoca.

Goma

Criatura verde menta dentro de um pote transparente com tampa de cortiça. Seu rosto fica em uma área sólida clara para não desaparecer com transparência. L1 recebe faixa; L2 ganha bocal. Poça tem dois bicos laterais. Supercola tem tampa reforçada e uma gota grande no topo. Transparência do pote é desenhada por áreas, evitando camadas de alpha custosas.

Voltz

Robô ciano de cabeça arredondada, rosto em visor escuro, corpo de mola e antena. L1 adiciona um anel; L2 alarga a bobina. Tempestade divide a antena em três pontas arredondadas. Alta tensão usa uma única esfera grande e bobina mais grossa. O brilho nunca deve esconder os olhos nem copiar o formato de um inimigo.

Maestro

Caranguejo coral com rádio, dois alto-falantes e chave de corda. L0 tem caixas pequenas; L1 ganha botão frontal; L2 amplia as caixas. Orquestra abre os alto-falantes para os lados. Solo usa corneta central compacta. Movimento de pinças marca o ritmo da aura. Sua animação de apoio ocupa o lugar da animação de ataque.

Inimigos e chefes

Entidade	Silhueta e sinais	Animação de leitura
Fiapo	Pequena bola violeta com dois olhos	Saltos curtos
Corrisco	Gota estreita com rastro curto	Corrida inclinada
Bolota	Corpo redondo largo e pesado	Compressão lenta
Latinha	Poeira dentro de lata quadrada	Passos duros e escudo visível
Névoa	Espiral achatada com borda pontilhada	Ondulação suave
Remendo	Saquinho com costura em cruz	Pulso de costura ao curar
Casulo	Casca oval rachada	Tremor antes de se dividir
Brutamontes	Corpo largo e dois braços de pano	Passos lentos e ombros altos
Aspirador	Corpo oval, mangueira e cara irritada	Inspiração de 2 s antes da poeira
Rei Ferrugem	Lata alta com coroa de tampinhas	Placas se fecham antes da armadura
Breu	Cobertor estrelado com olhos e pernas curtas	Costura brilha antes de invocar

Vida, resistência e habilidade nunca são comunicadas apenas pela aparência. Chefes têm barra própria e ícone de estado. O desenho de Breu mantém humor e expressão; evitar horror realista. Fiapos gerados por Casulo são menores e não exibem símbolo de recompensa.

Cenários e composição

Os três mapas usam exatamente as coordenadas do catálogo. O chão é visualmente contínuo, enquanto o caminho tem borda clara e pequenas setas nas curvas. Células de construção ficam discretas e aparecem melhor durante posicionamento. Props decorativos nunca invadem o corredor interativo de uma célula válida.

Jardim de Papel usa gramado teal, caminho de areia clara, plantas de papel e pedras de papelão. Predominam verde, creme e coral. O Farol de Pilha é pequeno, amarelo e coral, com janela iluminada. Entrada tem seta clara; saída tem conexão física com o farol.

Oficina de Lata usa bancada azul acinzentada, fita adesiva creme como rota, parafusos e caixas nas bordas. A estrada continua mais clara que o fundo. Não usar grades decorativas que possam ser confundidas com a grade de construção.

Sótão das Estrelas usa madeira escura dessaturada, tecido azul e estrelas de papel. O caminho continua claro mesmo com ambiente noturno. Pontos de luz pertencem à decoração; alertas de combate usam outra forma e movimento. Nenhum efeito de escuridão pode esconder posicionamento ou vida.

Interface de combate

O wireframe mostra hierarquia e distribuição, não arte pronta do jogo. As letras dentro das peças são rótulos para a documentação; serão substituídas por sprites e retratos. Dimensões do produto se adaptam ao dispositivo por BoardTransform.



Wireframe do combate com caminho e Circuitos

HUD usa painéis creme, cantos de 12 px e contorno ink de 2 px na escala de referência de 1280 px. Botão primário é gold com texto ink. Botão secundário usa cream. Danger aparece em venda e saída com confirmação explícita. Estado indisponível preserva texto legível e explica a razão, como “Faltam 120 sucatas”.

Tipografia de interface: Fredoka para nomes e títulos curtos, e família sem serifa do catálogo Roblox disponível no ambiente para números e textos longos. Verificar disponibilidade antes de fixar FontFace; fallback é Builder Sans ou fonte nativa equivalente verificada. Não incorporar fonte externa sem licença. No produto, tamanhos são metas após escala: corpo 16 a 18 px, números principais 20 a 24 px, títulos 28 a 36 px.

No celular em paisagem, reduzir decoração do HUD, recolher painel de detalhes e manter atalhos no polegar. Testar pelo menos 844×390, 896×414, 1280×720 e 1920×1080, mais um tablet 4:3. Respeitar safe areas. Em orientação vertical, mostrar orientação para girar o dispositivo e manter acesso a sair e configurações; não anunciar gameplay vertical otimizado.

Movimento e efeitos

Ação	Duração proposta	Direção
Hover ou foco	100 ms	Escala até 1,03
Clique confirmado	80 ms	Pequena compressão e retorno
Construção	180 ms	Entrada por escala sem deslocar célula
Melhoria	350 ms	Silhueta troca no meio do brilho
Disparo comum	200 a 330 ms	Antecipação curta e recuo
Impacto	120 a 220 ms	Ponto de contato e partícula curta
Morte comum	250 a 400 ms	Poeira se dissolve
Aviso de chefe	2 s	Contorno e ícone com pulsação lenta

Animações de sprites usam 12 fps como base, independentes do passo de simulação. Disparo desenhado pode ser abreviado em 2x. Modo de movimento reduzido desliga tremor de tela, saltos de painel e pulsação decorativa; avisos continuam por contorno estático e texto. Nenhum flash em tela inteira é necessário.

Circuitos usam fio fino e pontos de conexão sob as unidades. Mostrar todos discretamente; dar ênfase apenas à seleção e a uma nova ativação. Dano flutuante agrega impactos da mesma torre no mesmo alvo em janelas curtas, é opcional e tem limite de 30 números visíveis. Auras não precisam redesenhar círculos grandes permanentemente.

Áudio

Criar quatro loops musicais: menu e um por mapa, de 60 a 90 s, emenda sem clique, volume abaixo dos avisos. Instrumentos de brinquedo, percussão de madeira e sintetizador leve, sem imitar melodias de franquias. Chefes podem acrescentar uma camada de tensão ao loop existente.

Pacote mínimo de 24 efeitos: seis disparos ou apoios de torre; três impactos; três aparições ou habilidades de chefe; quatro ações de UI; três avisos de partida; três eventos de resultado ou desbloqueio; dois sons de base. Normalizar volume relativo e limitar repetição dos ataques. Avisos importantes têm equivalente visual. Registrar origem e direito de uso de cada áudio no manifesto.

Especificação de assets

Grupo	Quantidade inicial	Entrega
Torres	6 personagens × 5 estados	30 poses entregues; 240 quadros finais ainda a produzir
Inimigos comuns	8 × 12 quadros	96 quadros
Chefes	3 × 16 quadros	48 quadros
Cenários	3	Fundo, caminho e props separados
Retratos	6 base e variantes de skin	Recortes quadrados com área segura
Interface	24 ícones funcionais	Fonte vetorial e PNG exportado
Efeitos	12 famílias	Pequenos atlas ou formas nativas
Skins Fundador	6 × 5 estados	Variação visual sem mudar silhueta de função
Skins de domínio	6 × 5 estados	Variação visual após 250 pontos
Divulgação	1 ícone e 2 thumbnails	Composição própria após validar o produto

As skins podem reutilizar animação e estrutura dos estados, mas exigem revisão em todos os quadros. Não contabilizá-las como custo artístico zero. Adesivos de domínio: seis. Títulos: seis de domínio e um Fundador. Estandartes: padrão e Fundador. A produção pode começar sem a loja para priorizar o elenco base.

Quadros e atlas

Torres: originais estáticos entregues em 1254×1254; normalizar fonte de quadro para 512×512 e export de jogo para 128×128 RGBA durante a integração. Um atlas de 1024×1024 por torre, com oito colunas. Linhas 0 a 4 correspondem a L0, L1, L2, L3A e L3B. Colunas 0 a 3 são idle; 4 a 7 são ataque ou apoio. Restante transparente. A área desenhada fica dentro de 112×112, com margem para evitar sangramento. Posição dos pés igual em todos os quadros.

Inimigos comuns: quadro de 128×128, seis frames de caminhada, dois de impacto e quatro de morte. Dois atlas 1024×1024 comportam quatro inimigos cada, usando duas linhas de oito células por inimigo; as últimas quatro células do bloco ficam transparentes. Chefes: quadro de 256×256, oito frames de movimento, quatro de habilidade e quatro de morte; cada chefe usa um atlas 1024×1024 com 16 células.

Cada frame usa ImageRectOffset calculado por coluna e linha e ImageRectSize igual ao tamanho do quadro. Não usar margem externa que altere essa conta. Não espelhar sprites inteiros com letras ou emblemas assimétricos. A versão 1 usa uma pose principal; a direção do alvo é comunicada pelo projétil e um pequeno recuo, evitando girar todo o personagem de cabeça para baixo.

Fundos: os originais entregues medem 1586×992; preparar export de 1600×1000 por mapa, desenho limpo em 16:10, com camada de caminho separada e máscara de construção em dados. Interface usa PNGs pequenos ou elementos nativos. Manter fontes vetoriais ou arquivos em camadas fora do lugar Roblox. Asset final deve existir no repositório e no manifesto, sem depender do histórico de uma ferramenta de imagem.

Um atlas RGBA 1024×1024 ocupa aproximadamente 4 MiB quando decodificado sem considerar overhead. Seis de torres, dois de inimigos e três de chefes representam cerca de 44 MiB. Carregar somente o chefe e as skins usados na partida; reservar até 96 MiB para texturas próprias, incluindo variantes dos quatro jogadores, cenário e interface. Validar o pior caso em aparelhos reais. Tamanho do PNG comprimido não é medida suficiente de memória.

Nomes e manifesto

Padrão: tower_dardo_default_atlas_v001.png, enemy_common_a_atlas_v001.png, boss_aspirador_atlas_v001.png, map_jardim_background_v001.png e ui_currency_scrap_v001.png. Usar ASCII nos nomes técnicos, IDs estáveis e versões explícitas. Não sobrescrever uma versão já usada em produção sem registrar a mudança.

Manifesto exige assetKey, localFile, robloxAssetId, width, height, frameWidth, frameHeight, anchor, author, usageRights, moderationStatus e version. robloxAssetId fica vazio até upload real. Se uma imagem for rejeitada ou não carregar, apresentar fallback legível e registrar erro; a partida não pode ficar invisível.

Divulgação e consistência

Ícone inicial proposto: Dardo em primeiro plano e o Farol de Pilha ao fundo, com um Fiapo indicando ameaça. Thumbnail 1 mostra um Circuito entre dois personagens; thumbnail 2 mostra quatro jogadores por retratos defendendo contra Aspirador. Usar captura real quando o jogo existir e separar qualquer ilustração promocional. Não prometer funcionalidades ausentes.

Antes do upload, verificar as dimensões e regras vigentes do Creator Hub. Os arquivos mestres podem ser quadrados e 16:9 em alta resolução, mas o formato final deve seguir os requisitos reais da plataforma.

Aprovação visual

Uma torre é aprovada quando sua silhueta é reconhecível a 32 px, a função aparece a 48 px e os cinco estados são distintos sem depender do texto. Uma skin mantém esses critérios. Um mapa é aprovado quando caminho, entrada, saída e célula inválida são reconhecidos em até três segundos de observação no celular.

Revisar combate cheio em 1x e 2x, com partículas reduzidas e em escala de cinza. Confirmar que ícones de resistência sobrevivem à redução, o chefe não esconde a base, a interface cabe nas áreas seguras e todo botão oferece resposta de foco e toque. Aprovar primeiro Dardo completo e uma cena do Jardim como amostra de qualidade; depois produzir o restante do elenco com a mesma régua.

4 Testes e aceite

Estado desta entrega

Este pacote contém especificações, dados iniciais, referências e ilustrações. Os testes abaixo são obrigações para a implementação futura. Não foram executados contra um jogo Roblox nesta etapa. A validação do pacote cobre integridade de dados e apresentação dos documentos.

Testes das regras

ID	Cenário	Resultado esperado
T01	Colocar Dardo com 650 sucatas	Saldo 400 e uma torre L0
T02	Repetir requestId da mesma colocação	Mesmo resultado sem nova torre ou gasto
T03	Dois jogadores comprem na mesma célula	Apenas a primeira compra válida é aceita
T04	Colocar sobre caminho, bloqueio ou fora do mapa	Pedido rejeitado sem gasto
T05	Comprar L2 diretamente de L0	INVALID_STATE
T06	Dardo L0 e L1 são comprados e depois vendidos	Investido 450, devolução 315
T07	Dardo L3B atinge Latinha sem outros efeitos	Armadura física ignorada integralmente
T08	Goma e Pulso aplicam slow ao mesmo inimigo	Maior slow ativo prevalece, sem soma
T09	Supercola aplica 45% a chefe	Redução de velocidade 15,75%
T10	Dois Maestros de 12% e 25% cobrem Dardo	Intervalo dividido por 1,25
T11	Três torres disputam um Circuito	Pares determinísticos por ID, sem dupla conexão
T12	Vender parceiro conectado	Bônus removido no mesmo passo
T13	Voltz L0 acerta dois alvos sem resistência	Dano 20 no primeiro e 15 no segundo
T14	Casulo morre e cria três Fiapos	Um bounty de Casulo; zero dos filhos
T15	Casulo vaza	Um leak de Casulo, nenhum filho
T16	Chefe e base seriam eliminados no mesmo passo	Prevalece derrota pela ordem definida
T17	Dois jogadores usam Pulso simultaneamente	Um efeito e uma recarga
T18	Fiapo no Jardim Normal com quatro pessoas	Vida ceil($24 \times 3,1$) = 75
T19	Aspirador no Jardim Desafio solo	Vida ceil($7000 \times 1,45$) = 10150
T20	Bounty de 10 com três participantes	Distribuir 4,3,3 com cursor giratório

Persistência e resistência a abuso

ID	Cenário	Resultado esperado
P01	DataStore indisponível na entrada	Não sobrescrever perfil; treino sem progresso disponível
P02	Duas sessões disputam perfil	Uma adquire lock; outra não grava
P03	Servidor perde token de lock	Mutação persistente suspensa
P04	Falha de gravação após resultado	Mostrar pendente e reconciliar sem duplicação
P05	Repetir o mesmo matchId de recompensa	Nenhum novo pagamento

ID	Cenário	Resultado esperado
P06	Reprocessar tutorial concluído	Não conceder os 180 botões novamente
P07	Perfil antigo é carregado duas vezes	Migração idempotente
P08	Cliente envia preço zero ou recompensa fabricada	Campos rejeitados e nenhum saldo alterado
P09	Enviar NaN, infinito, strings enormes e IDs de outro match	Rejeitar antes de processar
P10	Jogador tenta vender torre alheia	NOT_OWNER
P11	Passe cancelado ou ID ausente	Nenhuma skin paga concedida
P12	Encerrar servidor com perfis sujos	Tentativa controlada de salvar, com resultado registrado

Testes no Studio e dispositivos

Executar um, dois e quatro clientes, construindo, votando 2x e recebendo resultados. Com 150 a 250 ms de latência simulada, saldo não duplica, comandos mostram espera e sprites se reconciliam sem alterar a simulação.

Testar as 24 combinações de três mapas, duas dificuldades e quatro tamanhos de grupo, com uma execução completa de cada. Registrar equipe, compras, vazamentos, duração e Pulso. Incluir equipes iniciais e avançadas.

Testar carga com três partidas de quatro pessoas e doze partidas solo. Registrar hardware, build, frames, p95 de simulação, memória e tráfego. Repetir 60 minutos de criação e encerramento; crescimento contínuo de instâncias ou conexões é falha.

No celular, concluir tutorial, construir, cancelar, ampliar, melhorar, vender e ativar Pulso. Manter confirmação e menus nativos acessíveis. Repetir com movimento reduzido, números desligados e áudio mudo.

Balanceamento e diversão

O catálogo é um ponto de partida. Usar um simulador de regras ou partidas reproduzíveis para verificar que Dardo mais Goma consegue atravessar as ondas iniciais do Jardim sem depender de sorte. No Normal, a composição inicial deve ter um caminho plausível até vitória; unidades desbloqueadas oferecem alternativas e não corrigem um bloqueio artificial.

Comparar escolhas A e B de cada torre em situações adequadas às funções. Se uma opção for melhor em todos os cenários, mudar custos, alcance ou efeito. Medir a utilidade de Maestro pelo dano adicional produzido e de Goma por inimigos contidos, além do dano bruto. Verificar se comprar Circuito compensa o custo de posição sem tornar obrigatório empilhar todas as torres.

Medir duração no Normal 1x: meta de 10 a 15 minutos. Se sessões mais curtas forem melhores, atualizar a promessa e a economia, sem esticar ondas artificialmente.

Definição de pronto

Todos os sistemas obrigatórios operam do início ao resultado; não há botão anunciado sem função. Dados e arte são carregados por manifesto; referências temporárias estão identificadas e removidas da versão final. Build abre, regras passam nos testes automatizados, partidas reais foram executadas e o relatório diferencia testado, não testado e bloqueado.

Lançamento exige ainda assets aprovados, descrição compatível com o produto, configuração de idade e conteúdo conforme o processo vigente do Roblox, dados de produção separados dos testes e opção de desativar a loja. Um problema de monetização não deve impedir jogar gratuitamente.

5 Instruções para Claude Code

Abra a pasta Tower Defense completa no Claude Code e copie a mensagem abaixo. Mantenha documentos, imagens e índices na organização entregue. A pasta contém documentação e recursos de arte; ainda não contém o jogo Roblox implementado.

Mensagem para iniciar

Você é responsável por implementar o jogo Roblox Quintal em Guarda a partir deste pacote. Construa a versão 1 completa descrita, em Luau com tipagem estrita e Rojo. Trabalhe no repositório atual, preserve alterações existentes e leia instruções locais antes de editar.

Na raiz Tower Defense, leia primeiro 00_COMECE_AQUI.md e CLAUDE.md. Dentro de Pacote_Claude_Code, leia 00_LEIA_PRIMEIRO.md, 01_GAME_DESIGN.md, 02_ESPECIFICACAO_TECNICA.md, 03_DIRECAO_DE_ARTE.md, 04_TESTES_E_ACEITE.md, 06_CATALOGO_NUMERICO.md, 08_USO_DE_ASSETS_E_ANIMACOES.md e dados/balanceamento_v1.json. Leia também asset_index.json, runtime_asset_registry.json e animation_plan.json na raiz. Consulte 07_FONTES.md quando precisar confirmar uma API. Veja as imagens de arte antes de implementar a interface. O JSON é canônico para números; os textos definem semântica, experiência e qualidade visual. Identifique conflitos concretos antes de alterar essas decisões.

Entregue gameplay realmente 2D por ScreenGui, com simulação autoritativa no servidor, caminho fixo e sprites. Não converta o produto em mundo 3D ou apenas em protótipo de interface. O produto obrigatório inclui tutorial, treino, seis torres e seus trinta estados, três Circuitos, oito inimigos, três chefes, três mapas, vinte ondas, duas dificuldades, Pulso de Luz, cooperação de até quatro pessoas, coleção, domínio, persistência, resultado, revanche e configurações. A loja de passe cosmético deve poder permanecer desativada sem afetar o jogo.

Comece inspecionando ferramentas disponíveis, o estado do repositório e acesso ao Studio. Registre um plano por marcos M0 a M6 com critérios de verificação, e então implemente. Não pare depois de propor o plano, desenhar telas ou terminar a primeira fase. Continue até concluir o escopo implementável e verificar as partes acessíveis. Se uma dependência externa bloquear algo, registre a pendência específica e avance nas demais partes independentes.

Crie um projeto que possa ser sincronizado com Rojo e aberto no Roblox Studio, com dependências fixadas e instruções reproduzíveis. Gere os módulos de configuração a partir do JSON e escreva validações para IDs, custos, ondas e caminhos. Use módulos de regras puras para combate, alcance, Circuitos e recompensas. Não coloque economia ou decisão de dano no cliente. Nenhum RemoteEvent pode aceitar dano, preço ou resultado arbitrário enviado pelo jogador.

Construa primeiro uma partida curta do início à vitória e derrota; depois complete as torres, Circuitos, conteúdo e cooperação. Implemente as regras completas, incluindo empate, acumulação de efeitos, dono, limites, desconexão e duplicação de pedidos. Não substitua funcionalidades obrigatórias por TODOs silenciosos. Evite sistemas extras fora do escopo e abstrações que não ajudem o produto.

Use o catálogo como balanceamento inicial. Execute testes das regras e documente alterações justificadas por simulação ou playtest. Não declare que os números estão equilibrados sem evidência. Preserve a viabilidade da equipe inicial e a diferença entre especializações. Qualquer mudança numérica deve atualizar o JSON, os módulos gerados e os testes relevantes.

Implemente o fluxo visual usando os 40 PNGs existentes em Assets_Quintal_em_Guarda_v1, localizados por asset_index.json. São 30 estados estáticos de torres, três terrenos e sete imagens de cenário e base. Preserve os originais, prepare exports, valide apoios e preencha IDs somente após upload real. Implemente movimentos procedurais conforme animation_plan.json e capítulo 8, sem confundi-los com atlas finais. Produza depois os quadros de animação, inimigos, chefes e demais assets ausentes. Se uploads ou ferramentas de arte estiverem indisponíveis, mantenha a integração preparada, use visuais provisórios identificados somente onde necessário e registre as pendências.

Nunca invente IDs Roblox, sons licenciados, passes, experiência publicada, credenciais ou resultado de moderação. Configure placeholders tipados e desligamento seguro para recursos externos. Guarde scripts e documentação no repositório. Quando houver ferramentas disponíveis, produza o build de lugar e registre o caminho real.

Valide com testes automatizados de regras, análise estática e testes reais no Studio. Priorize os cenários de 04_TESTES_E_ACEITE.md, incluindo compras simultâneas, recompensa duplicada, DataStore indisponível, quatro clientes e interface de celular. Se não puder executar Studio, mantenha uma lista explícita dos testes não executados e das ações necessárias; não use essa limitação como motivo para deixar módulos implementáveis incompletos.

Ao concluir cada marco, registre o que mudou, o que foi verificado e o que falta. Ao concluir a execução, entregue código, estrutura Rojo, dados, assets disponíveis, manifesto, SETUP.md, TEST_REPORT.md e MANUAL_ACTIONS.md. Diferencie claramente código implementado, experiência testada, arte integrada e publicação. Não publique nem efetue compras em conta externa sem autorização do responsável.

Considere o objetivo concluído somente quando o escopo obrigatório implementável estiver entregue, as verificações disponíveis tiverem sido executadas e qualquer pendência externa estiver descrita de forma acionável. O relatório final deve citar falhas reais e limitações, sem afirmar que uma experiência comercial completa foi validada apenas porque os arquivos foram gerados.

Instrução para retomar uma execução interrompida

Leia o pacote de especificação e o registro de marcos do repositório. Verifique o estado atual do código e o último relatório de testes. Continue do primeiro requisito obrigatório incompleto; não recomece o projeto nem substitua regras já implementadas sem motivo. Atualize testes e documentação junto com cada mudança e prossiga até a conclusão do escopo acessível.

6 Catálogo numérico

Valores iniciais propostos, sem playtest. Custos de L1, L2 e L3 são incrementais; L3A e L3B são exclusivos. Dano por acerto, intervalo em segundos e alcance em células. Regras de efeitos e precedência estão na especificação técnica.

Dardo

ID dardo. Ataque direto. Desbloqueio: inicial. Máximo por jogador: 6.

Estado	Custo	Dano	Intervalo	Alcance	Efeito
L0	250	10	0.8	2.5	Acerto simples
L1	200	16	0.75	2.7	Acerto simples
L2	450	28	0.7	2.9	Acerto simples
L3A	900	46	0.55	3.1	Acerto simples
L3B	1000	42	0.75	3.4	armorIgnore=1

L3A: Rajada. L3B: Fura lata.

Pipoca

ID pipoca. Explosão em área. Desbloqueio: inicial. Máximo por jogador: 4.

Estado	Custo	Dano	Intervalo	Alcance	Efeito
L0	450	20	1.8	2.8	splash=1, maxTargets=6
L1	300	34	1.7	3	splash=1.1, maxTargets=6
L2	600	60	1.6	3.2	splash=1.2, maxTargets=8
L3A	1200	95	1.4	3.4	splash=1.7, maxTargets=10
L3B	1200	115	1.8	3.6	splash=1.1, maxTargets=6, armorIgnore=0.5

L3A: Festival. L3B: Demolidora.

Lupa

ID lupa. Dano contra elites. Desbloqueio: 240 botões. Máximo por jogador: 4.

Estado	Custo	Dano	Intervalo	Alcance	Efeito
L0	500	55	2.5	5	Acerto simples
L1	350	85	2.3	5.3	Acerto simples
L2	700	145	2.1	5.6	Acerto simples
L3A	1400	280	2.2	6	bossBonus=0.25
L3B	1300	170	1.6	6	mark=0.1, markDuration=3

L3A: Olho de águia. L3B: Observadora.

Goma

ID goma. Lentidão. Desbloqueio: inicial. Máximo por jogador: 4.

Estado	Custo	Dano	Intervalo	Alcance	Efeito
L0	300	3	1	2.3	slow=0.2, slowDuration=1.5
L1	200	5	1	2.5	slow=0.25, slowDuration=1.8
L2	400	8	0.9	2.7	slow=0.3, slowDuration=2
L3A	850	10	0.9	3	slow=0.3, slowDuration=2, splash=1, maxTargets=5
L3B	850	12	0.8	3	slow=0.45, slowDuration=2.5

L3A: Poça. L3B: Supercola.

Voltz

ID voltz. Corrente elétrica. Desbloqueio: 600 botões. Máximo por jogador: 4.

Estado	Custo	Dano	Intervalo	Alcance	Efeito
L0	550	20	1.3	2.4	chains=2, chainRadius=1.5, chainFalloff=0.75
L1	350	30	1.2	2.6	chains=3, chainRadius=1.5, chainFalloff=0.75
L2	650	48	1.1	2.8	chains=3, chainRadius=1.5, chainFalloff=0.75
L3A	1250	60	1	3	chains=5, chainRadius=1.7, chainFalloff=0.8
L3B	1250	90	1.15	3	chains=2, chainRadius=1.5, chainFalloff=0.75, bossBonus=0.2

L3A: Tempestade. L3B: Alta tensão.

Maestro

ID maestro. Apoio de velocidade. Desbloqueio: 450 botões. Máximo por jogador: 2.

Estado	Custo	Dano	Intervalo	Alcance	Efeito
L0	500	0	0	2.2	haste=0.08
L1	300	0	0	2.4	haste=0.12
L2	600	0	0	2.6	haste=0.16
L3A	1150	0	0	3.2	haste=0.18
L3B	1150	0	0	2.6	haste=0.25

L3A: Orquestra. L3B: Solo.

Inimigos comuns

ID	Vida	Velocidade	Sucata	Dano na base	Armadura
fiapo	24	1.25	8	2	0
corrisco	32	2.05	10	3	0
bolota	150	0.85	22	6	0
latinha	115	1	20	5	0.45
nevoa	75	1.45	17	4	0
remendo	180	1	28	6	0
casulo	230	0.9	30	8	0.15
brutamontes	600	0.65	65	14	0.25

Chefes

A vida abaixo já corresponde ao chefe do seu mapa. Aplicar somente cooperação e dificuldade, sem multiplicador adicional do mapa. Todos causam 100 de dano ao vazar, não dão sucata própria e recebem apenas 35% da força normal de lentidão.

Chefe	Vida	Velocidade	Armadura	Período da habilidade
Aspirador Rabugento	7000	0.4	0.15	14 s
Rei Ferrugem	8500	0.4	0.2	14 s
Breu debaixo da cama	10000	0.4	0.1	16 s

Roteiro das ondas

Grupos aparecem na ordem da tabela. Há dois segundos entre grupos. O chefe entra dois segundos após o último grupo da onda 20. Intervalos: 1,1 s nas ondas 1 a 4; 0,85 s nas ondas 5 a 10; 0,65 s nas ondas 11 a 20.

Onda	Composição	Sucata por jogador ao concluir
1	12 fiapo	135
2	16 fiapo	150
3	12 fiapo; 6 corrisco	165
4	20 fiapo; 2 bolota	180
5	12 corrisco; 4 bolota	195
6	20 fiapo; 6 latinha	210
7	16 corrisco; 8 latinha	225
8	6 bolota; 12 nevoa	240
9	24 fiapo; 10 latinha; 2 remendo	255
10	20 corrisco; 8 bolota; 3 remendo	270
11	18 nevoa; 5 casulo	285
12	16 latinha; 6 casulo	300
13	24 corrisco; 2 brutamontes; 4 remendo	315
14	20 nevoa; 18 latinha; 6 casulo	330

Onda	Composição	Sucata por jogador ao concluir
15	4 brutamontes; 5 remendo; 12 bolota	345
16	30 corrisco; 10 casulo; 18 latinha	360
17	6 brutamontes; 24 nevoa; 6 remendo	375
18	25 latinha; 12 casulo; 5 brutamontes	390
19	32 corrisco; 8 brutamontes; 7 remendo	405
20	20 fiapo; 14 latinha; 4 remendo	420

Coordenadas dos mapas

Coordenadas inteiras de centro de célula, origem no canto superior esquerdo. Todos os segmentos são ortogonais. O caminho ocupa cada célula atravessada, inclusive as extremidades. Células bloqueadas são adicionais ao caminho.

Jardim de Papel

ID jardim. Multiplicador de vida dos inimigos comuns: 1. Chefe: aspirador.

Caminho: [[0, 2], [4, 2], [4, 6], [9, 6], [9, 3], [13, 3], [13, 8], [15, 8]].

Bloqueios: [[1, 8], [2, 8], [11, 0], [12, 0], [0, 9]].

Oficina de Lata

ID oficina. Multiplicador de vida dos inimigos comuns: 1.12. Chefe: rei_ferrugem.

Caminho: [[0, 7], [3, 7], [3, 2], [7, 2], [7, 7], [11, 7], [11, 3], [15, 3]].

Bloqueios: [[0, 0], [1, 0], [14, 9], [15, 9], [8, 9]].

Sótão das Estrelas

ID sotaos. Multiplicador de vida dos inimigos comuns: 1.25. Chefe: breu.

Caminho: [[0, 1], [5, 1], [5, 5], [2, 5], [2, 8], [10, 8], [10, 2], [14, 2], [14, 6], [15, 6]].

Bloqueios: [[8, 0], [9, 0], [6, 9], [7, 9], [15, 0]].

7 Fontes e pesquisa

Consulta realizada em 15 de setembro de 2026. Foram usadas páginas oficiais para confirmar identidade e mecânicas anunciadas; as decisões para o novo jogo são propostas próprias. Não foram usadas estatísticas internas, sessões de jogo próprias ou entrevistas. Este documento não estabelece ranking absoluto de jogadores simultâneos.

Jogos do Roblox

1. [Tower Defense Simulator de Paradoxum Games](#). Unidades, cooperação, chefes e anúncio da versão 2.10.0 observados na consulta.
2. [Tower Defense X](#). Torres em diferentes ambientes, ameaças às torres e anúncio de PvP.
3. [Tower Heroes de Pixel bit Studio](#). Heróis, mapas com chefes, coleção e modos cooperativos.
4. [Anime Vanguard de Kitawari](#). Invocação, evolução de unidades e modos com amigos.
5. [Doomspire Defense de VY BOX](#). Tema, recompensas, compra de torres e composições.

Os nomes e títulos das páginas Roblox podem mudar com eventos. Os IDs numéricos identificam as experiências usadas. Textos genéricos sobre ausência de servidores em páginas sem JavaScript não comprovam falta de atividade. Contadores indexados não foram usados como fotografia de audiência ao vivo.

Referências de estratégia e apresentação

6. [Bloons TD 6 na Steam pela Ninja Kiwi](#). Caminhos de evolução, heróis, desafios e cooperação.
7. [Perguntas e respostas sobre Kingdom Rush Alliance pela Ironhide](#). Uso de dois heróis e estrutura de campanha. Fonte histórica de design de um jogo disponível na consulta; não foi tratada como atualização de setembro de 2026.
8. [Emberward na Steam pela Refic Games](#). Construção de labirintos, blocos e posicionamento de torres. Na página consultada, ainda aparecia em acesso antecipado, com saída anunciada para 17 de setembro de 2026. Uma data anunciada não foi tratada como lançamento já ocorrido.

Documentação técnica primária

9. [Contêineres de interface na tela](#). ScreenGui, objetos locais e áreas seguras.
10. [ImageLabel](#). Propriedades de imagem e recorte para atlas.
11. [Segurança na fronteira cliente servidor](#). Validação de entradas, permissões e limites.
12. [Data stores](#). Persistência, UpdateAsync e cuidados com dados de teste.
13. [Passes](#). Compra permanente e integração de posse.
14. [Developer products](#). Referência para eventual expansão e processamento de recibos.
15. [Eventos de funil](#). Instrumentação de progressão na experiência.
16. [Modos de teste do Studio](#). Verificações do modelo cliente servidor.
17. [Introdução ao Rojo](#). Projeto em arquivos e integração de ferramentas.

Rever APIs e requisitos de publicação quando a implementação começar. A arquitetura detalhada, as taxas de atualização, os orçamentos, a economia e os critérios de teste são decisões deste projeto, não afirmações de que a documentação Roblox prescreve esses valores.

Questões para testar com jogadores

As referências apoiam escolher coleção, especialização e cooperação como pilares. Elas não provam que brinquedos, Circuitos, partidas de determinada duração ou a oferta cosmética terão aceitação. Essas quatro hipóteses precisam de protótipo jogável, observação e dados próprios. A análise evitou atribuir aos concorrentes falhas de retenção ou monetização não demonstradas pelas fontes.

8 Integração das imagens e animações

O que mudou na versão 1.1

O projeto agora acompanha 40 PNGs individuais produzidos a partir da direção de arte. São 30 visuais de torres, três terrenos, seis objetos de cenário e um Farol de Pilha. Use esses arquivos na implementação; a prancha conceitual continua sendo referência de estilo. Este capítulo define onde localizar a arte, como integrá-la, como animar as poses disponíveis e o que ainda falta produzir.

As imagens já foram salvas em arquivos, com canal alpha verificado. Ainda não houve upload, aprovação de moderação nem integração no Roblox Studio. Os PNGs são poses estáticas em alta resolução; não são os 240 quadros de torres previstos para a animação final. O plano artístico completo do capítulo 3 continua válido.

Pasta de trabalho e ordem de leitura

Abra no Claude Code a pasta Tower Defense indicada pelo responsável. Todos os caminhos abaixo são relativos a essa raiz; não dependa do caminho pessoal do computador, do histórico desta conversa nem de uma imagem anexada ao chat. A organização permite copiar a pasta para outro computador sem reescrever as referências.

```
Tower_Defense/
  00_COMECE_AQUI.md
  CLAUDE.md
  Quintal_em_Guarda_Documento_Completo.pdf
  Quintal_em_Guarda_Documento_Completo.docx
  Pacote_Claude_Code/
    00_LEIA_PRIMEIRO.md
    01_GAME_DESIGN.md
    02_ESPECIFICACAO_TECNICA.md
    03_DIRECAO_DE_ARTE.md
    04_TESTES_E_ACEITE.md
    05_PROMPT_PARA_CLAUDE_CODE.md
    06_CATALOGO_NUMERICO.md
    07_FONTES.md
    08_USO_DE_ASSETS_E_ANIMACOES.md
    dados/balanceamento_v1.json
    arte/
  Assets_Quintal_em_Guarda_v1/
    torres/
    fases/
    cenario/
    manifesto_assets.json
    CATALOGO.html
    PREVIA_FASES.html
    layout_fases.json
    LEIA_PRIMEIRO.md
    PROMPT_CLAUDE_INTEGRACAO.md
    prompts_geracao.json
    verificacao_arquivos.json
  asset_index.json
  runtime_asset_registry.json
  animation_plan.json
  INVENTARIO_ARQUIVOS.json
```

Comece por 00_COMECE_AQUI.md e CLAUDE.md. Leia a especificação de gameplay e técnica antes de modificar regras. Para usar a arte, leia este capítulo e asset_index.json. O catálogo HTML ajuda a escolher imagens; a prévia HTML demonstra composição, sem simular uma partida. Abra os HTMLs junto de suas subpastas, após o download completo dos arquivos.

Estado real da biblioteca

Grupo	Arquivos disponíveis	Uso imediato	Produção pendente
Torres	30 PNGs de 1254 × 1254	Repouso, melhoria, retrato e coleção	Quadros de repouso e ataque ou apoio
Terrenos	3 PNGs de 1586 × 992	Camada de chão de cada fase	Export final e ajuste da área do tabuleiro
Cenário e base	7 PNGs de 1254 × 1254	Decoração e Farol de Pilha	Efeitos opcionais e export final
Inimigos e chefes	Nenhum sprite neste lote	Usar visuais provisórios identificados	Oito inimigos e três chefes
Outros	Nenhum áudio ou skin final neste lote	Implementar pontos de integração	Áudio, efeitos, interface e cosméticos

Cada PNG tem transparência verdadeira ao redor da silhueta. O piso dos terrenos permanece opaco e sua margem externa é transparente. A Goma tem vidro pintado por áreas de cor; não deve ficar invisível ao aplicar transparência adicional. Não remover automaticamente fundos de arquivos que já possuem alpha.

Nomes e localização exata

Os nomes foram padronizados sem espaços nem acentos. Não renomear só para traduzir para a interface; o nome apresentado ao jogador vem da localização. Estado L0 é a base; L1 e L2 são melhorias sequenciais; L3A e L3B são alternativas finais exclusivas.

O padrão de torre é `Assets_Quintal_em_Guarda_v1/torres/tower_{id}_{estado}_v001.png`. Substitua `{id}` e `{estado}` pelos valores da tabela. Há exatamente cinco arquivos por torre.

ID da torre	Estados presentes	L3A	L3B
dardo	L0 L1 L2 L3A L3B	Rajada	Fura lata
pipoca	L0 L1 L2 L3A L3B	Festival	Demolidora
lupa	L0 L1 L2 L3A L3B	Olho de águia	Observadora
goma	L0 L1 L2 L3A L3B	Poça	Supercola
voltz	L0 L1 L2 L3A L3B	Tempestade	Alta tensão
maestro	L0 L1 L2 L3A L3B	Orquestra	Solo

Exemplo completo: `Assets_Quintal_em_Guarda_v1/torres/tower_dardo_L3A_v001.png`. Sua chave estável é `tower_dardo_L3A`. A extensão e o sufixo de versão pertencem ao arquivo; a chave é usada pelo jogo e pelo registro de upload.

Os terrenos ficam em fases, com os nomes `map_jardim_ground_v001.png`, `map_oficina_ground_v001.png` e `map_sotao_ground_v001.png`. Os objetos ficam em cenário. O inventário inclui todos os caminhos completos relativos à raiz, evitando depender de glob ou de ordem alfabética para decidir qual estado carregar.

Qual arquivo de dados usar

`asset_index.json` é o índice portátil para o Claude. Seus `sourceFile` partem da raiz Tower Defense. Ele associa cada imagem à torre, estado ou mapa e registra dimensões, checksum e ponto de apoio sugerido. `manifesto_assets.json` é o registro original da geração; seus `localFile` partem da pasta `Assets_Quintal_em_Guarda_v1`. Não concatenar essas duas bases por engano.

`runtime_asset_registry.json` é o modelo de registro de integração. Começa com os 40 IDs Roblox vazios e `status_pending_upload`. Preencha `exportFile`, `robloxAssetId`, `status` e dimensões efetivamente importadas somente quando existirem. Gere `Config/Assets.luau` desse registro e valide sua cobertura contra `asset_index.json`. Não alterar o manifesto original para fingir que uma imagem foi publicada.

`animation_plan.json` descreve movimentos, duração, comportamento em 2x e quadros futuros. Trata-se de configuração e especificação; não contém um animador implementado. As imagens continuam sendo os

mesmos 40 originais. O arquivo `INVENTARIO_ARQUIVOS.json` permite verificar que os documentos, imagens e dados foram copiados integralmente.

Preparação para renderização

Preserve os originais. Gere cópias em `assets/export` durante a implementação, sem gravar por cima de torres, fases ou cenário. Para sprites de torres, prepare fonte normalizada em 512×512 e export de 128×128 , como definido na direção de arte. A área desenhada deve respeitar a margem de segurança, com apoio em (0,5; 0,82) no quadro normalizado. Produza miniaturas maiores separadas se a coleção precisar delas.

Os originais têm enquadramentos diferentes. `anchorNormalizedSuggested` foi estimado pela região dos pés e precisa de revisão; `contentBoundsPx` considera pixels com alpha de pelo menos 128 e não é uma máscara de recorte. Preserve a suavização de bordas. Aplique uma transformação de escala uniforme e translação por arquivo, validada com os cinco estados lado a lado. Se um acessório não couber, reduza a família inteira ou ajuste explicitamente o enquadramento; não corte lunetas, orelhas ou alto-falantes.

Mantenha a escala aparente do corpo coerente entre níveis. Não normalize cada torre apenas pela largura total: a cauda de Dardo e os alto-falantes de Maestro fazem parte da silhueta. No código, o ponto lógico da célula pertence a um contêiner estável; a imagem é filha dele. Ajustes visuais e animações ocorrem nessa filha, sem mover a célula ou mudar a seleção.

Para os três terrenos, a pequena diferença entre a proporção original e 16:10 deve ser resolvida na exportação por enquadramento ou margem. Não inferir o caminho pelo desenho do piso. Mantenha um fundo sólido coerente abaixo da imagem para que a margem transparente não revele uma área branca ou o ambiente 3D.

Upload e carregamento no Roblox

Prepare e revise os exports, importe as imagens pela ferramenta de assets do Studio usando o proprietário da experiência, aguarde o resultado e registre os IDs reais. Verifique acesso na experiência de destino e teste o carregamento no cliente. Rojo sincroniza a estrutura de código, mas o projeto ainda precisa referenciar os conteúdos importados. Consulte o [Asset Manager](#) para o fluxo vigente.

Um sprite estático usa `ImageLabel` com fundo da interface transparente, imagem visível e proporção preservada. Não aplicar o recorte de um atlas a um PNG individual. Apenas exports em atlas usam `ImageRectOffset` e `ImageRectSize`. Essas propriedades estão na referência de [ImageLabel](#); os nomes de pastas, escalas e âncoras deste capítulo são decisões deste projeto.

Carregue apenas o mapa e as unidades necessários à partida, incluindo os estados de melhoria que podem ser usados. Pré-carregue um conjunto limitado, trate falhas e limite a espera de carregamento na interface. Mantenha um visual de reserva legível se o asset não estiver disponível e registre sua chave. A documentação de [ContentProvider](#) descreve o carregamento; não interprete download concluído como aprovação artística ou teste de desempenho.

Montagem das fases

Use o terreno correspondente ao `mapId`. Desenhe o caminho em uma camada separada a partir de `dados/balanceamento_v1.json`, com segmentos ortogonais, entrada, saída e largura coerentes com a grade de 16×10 . Mantenha as camadas de chão, caminho, Circuitos, sombras, entidades, efeitos e seleção nas faixas definidas na especificação técnica.

O Farol de Pilha representa a base na saída. Props decorativos ficam nas células bloqueadas ou fora da área interativa, sem alterar colisão ou esconder torres. `layout_fases.json` e `PREVIA_FASES.html` são exemplos de apresentação: seus caminhos reproduzem o catálogo, mas as posições de torres são demonstrativas. O `balanceamento_v1.json` continua prevalecendo para gameplay.

Movimento possível com as imagens atuais

É possível dar vida ao jogo imediatamente por escala, pequeno deslocamento e rotação do sprite inteiro. Isso não cria quadros desenhados nem permite mover isoladamente braços, olhos ou armas. Não deformar a identidade para simular articulações que não existem. Use animação procedural como apresentação inicial identificada e mantenha a troca futura para atlas desacoplada das regras.

As metas abaixo são parâmetros iniciais de movimento. Duração se refere ao tempo de simulação, exceto interações de interface. Normalize amplitudes pelo tamanho exibido e preserve os pés sempre que possível. [TweenService](#) oferece interpolação; a divisão em contêiner lógico e visual e os valores propostos são decisões do projeto.

Torre	Repouso com uma pose	Ataque ou apoio com uma pose
Dardo	Escala vertical até 1,015 em 1,6 s	Recuo visual de até 3 px em 128 px e retorno em 0,22 s
Pipoca	Respiração discreta até 1,015 em 2 s	Compressão vertical até 0,94 e retorno em 0,30 s
Lupa	Escala vertical até 1,008 em 2,4 s	Recuo de até 2 px e retorno em 0,24 s
Goma	Largura até 1,02 e altura até 0,98 em 1,8 s	Compressão até 0,95 e retorno em 0,28 s
Voltz	Escala vertical até 1,025 em 1,2 s	Compressão até 0,92 e retorno em 0,20 s
Maestro	Inclinação de até 1 grau em 1,4 s	Escala até 1,03 em 0,30 s, no máximo um pulso visual por segundo

Limite por padrão os movimentos aos sprites ativos na tela. Em movimento reduzido, desligue os ciclos contínuos, inclinação e deslocamento; mantenha mudança de estado e feedback curto no contorno. Não aplicar tremor global ou flashes de tela inteira. Sombras são elementos separados e não devem pulsar com a mesma intensidade do personagem.

Construção pode usar escala de 0,85 a 1 em 180 ms. Melhoria troca a imagem uma vez no meio de uma transição de 350 ms, preservando o apoio. Venda remove a representação depois da confirmação do servidor. Não use uma sequência L0 L1 L2 como animação de ataque: são estados de gameplay diferentes.

Autoridade e ciclo do animador

O servidor resolve aquisição de alvo, dano e cooldown conforme o capítulo técnico. O cliente recebe um evento cosmético de ataque com `entityId`, `attackSeq`, `simTime` e informação mínima dos alvos confirmados. Deduplicar por `entityId` e `attackSeq`. Mudar a imagem ou terminar um tween nunca causa dano nem concede recompensa.

Projéteis, correntes e impactos são apresentação. Como o dano é instantâneo na simulação, o projétil pode ser breve; uma morte confirmada não espera a imagem alcançá-la. Um cliente lento pode omitir efeitos antigos, preservando o estado correto. Não enviar um comando de dano ao servidor a partir da animação.

Use um controlador central no cliente com atualização por delta de tempo. Mantenha posição lógica, transformação visual e seleção independentes. Idle só atua quando não há ação de maior prioridade. Ao atacar, cancela ou suspende o idle; ao concluir, restaura escala 1, rotação 0 e deslocamento zero antes de retomá-lo. Melhoria substitui a textura e reinicia a apresentação; remoção cancela tudo e libera conexões e efeitos.

Em 2x, animações de combate acompanham o relógio de simulação e ficam mais curtas em tempo real; cliques e painéis mantêm duração de interface. Se usar tweens de tempo real, calcule a duração efetiva e reconcilie ao mudar a velocidade. Um controlador por `simTime` facilita essa consistência. Não empilhe centenas de tweens, `task.waits` ou conexões por torre.

Plano para animação desenhada

Produza quatro quadros de repouso e quatro de ataque ou apoio para cada estado. Isso corresponde a oito quadros por estado, 40 por personagem e 240 para as seis torres. As 30 imagens entregues são guias de identidade e pose, não 30 atlas. Não preencher quadros futuros com cópias iguais e declarar a animação concluída.

Colunas no atlas	Conteúdo	Reprodução
0 a 3	Repouso	Ciclo de quatro quadros
4 a 7	Ataque ou apoio	Uma passagem e retorno ao repouso

As linhas 0 a 4 correspondem a L0, L1, L2, L3A e L3B. Um atlas de 1024×1024 com quadros de 128×128 tem oito colunas e oito linhas; as três linhas finais permanecem transparentes. Cada personagem recebe um atlas. A fórmula de offset é $\text{coluna} \times 128 + \text{linha} \times 128$, sem borda externa que altere a conta. Preserve margens internas, apoio e proporção em todos os quadros.

Use 12 fps como referência de leitura. Quatro quadros duram aproximadamente 333 ms; ajuste a duração de ataque ou o fps da ação para coincidir com o movimento aprovado, sem mudar o intervalo de dano. O manifesto de runtime deverá mudar de `single_static` para atlas apenas depois que o arquivo de atlas existir e seus recortes forem validados.

Para cada torre, desenhe antecipação, ação, recuperação e repouso coerentes com sua função. Dardo recua o disparador; Pipoca comprime o casco; Lupa ajusta a luneta; Goma movimenta a massa dentro do pote; Voltz comprime a mola; Maestro movimenta pinças e alto-falantes. Quando esses movimentos exigirem partes internas, produza quadros ou camadas próprios, mantendo o contorno e os materiais existentes.

Inimigos e chefes seguem o plano do capítulo 3: 96 quadros para os oito inimigos e 48 para os três chefes. Nada desse grupo foi incluído neste lote. Skins, efeitos e áudio também precisam de produção e aprovação próprias.

Critérios de integração concluída

A integração inicial está pronta quando os 40 arquivos são localizados pelo índice, cada estado mostra a imagem correta, os apoios não saltam ao melhorar, as três fases respeitam o caminho canônico e a transparência funciona sobre os três terrenos. Verifique os sprites a 128 px e no tamanho efetivo do celular, inclusive as especializações mais largas.

Teste compra, melhoria, venda, ataque durante melhoria, mudança 1x para 2x, movimento reduzido, encerramento de partida, reuso do pool e falha de imagem. Nenhuma dessas ações pode duplicar dano, deixar animação presa ou acumular conexões. Perfis de desempenho devem usar os exports reais; tamanho comprimido dos PNGs não substitui medida de memória.

Registre separadamente arte importada, movimentos procedurais implementados, atlas produzidos e testes executados no Studio. A definição de pronto para animação final exige os quadros desenhados previstos e sua revisão em jogo. Um relatório deve informar pendências concretas, sem declarar animação final apenas porque a pose estática se moveu.